University Research and Lab Asset Tracker

Phase 3

Nhóm BPT | Môn Cơ sở dữ liệu | Học kỳ 1 năm 2026 2027

4 Triển khai cơ sở dữ liệu

4.1 Mục tiêu và môi trường triển khai

Phase 3 chuyển lược đồ logic của Phase 2 thành cơ sở dữ liệu thực thi, nạp bộ dữ liệu mô phỏng, thực hiện các truy vấn quản trị và kiểm chứng ràng buộc nghiệp vụ. Tên bảng dùng lower_snake_case. Giữ nguyên khóa rearch_point_id để khớp tài liệu gốc và tránh đổi định danh giữa các phase.

Mã SQL hướng tới PostgreSQL, dùng PL/pgSQL, khóa hàng, constraint trigger trì hoãn, partial unique index và SECURITY DEFINER. Phiên xác minh thực tế chạy ngày 21/10/2026 trên PostgreSQL 18.6 qua PGlite 0.5.10. Kết quả này xác nhận thực thi SQL trong một kết nối; không thay thế kiểm thử đồng thời hai phiên trên PostgreSQL máy chủ.

Bộ script tạo 19 bảng gồm 16 bảng nền của Phase 2 và 3 bảng bổ sung có phạm vi rõ ràng; tạo 5 view và 23 trigger người dùng. Dữ liệu mẫu hoàn toàn tổng hợp, không dùng thông tin cá nhân hoặc hóa chất tồn thật.

4.1.1 Hoàn thiện lược đồ vật lý

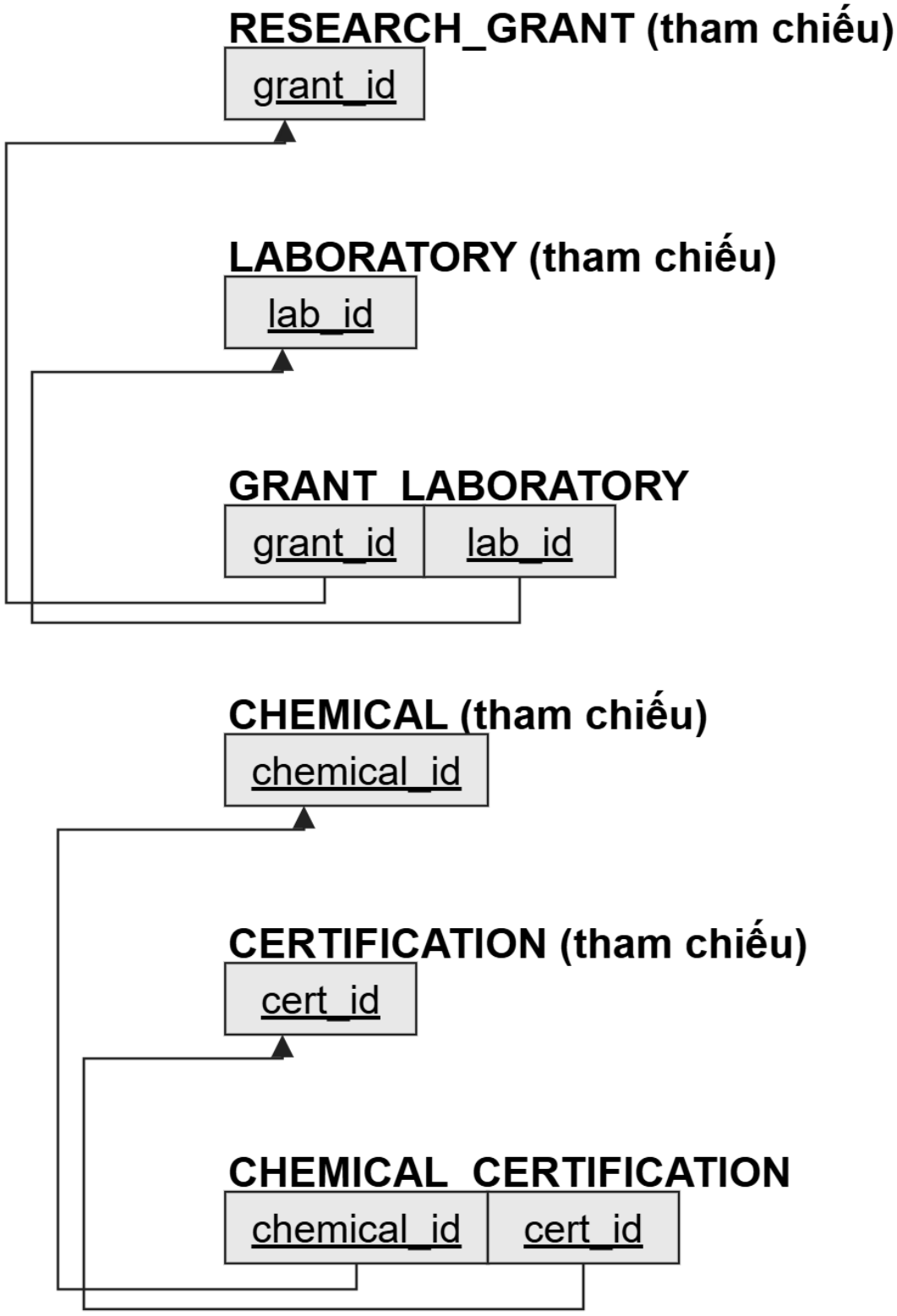
Phase 2 chỉ lưu liên kết tài sản–hạng mục, chưa có dữ liệu phòng được cấp cho đề tài, tập chứng nhận phù hợp với từng hóa chất và sổ biến động tồn kho. Ba bảng bổ sung lấp đúng ba khoảng trống này. Không thêm thuộc tính số lượng hoặc chi phí vào hai bảng allocated_equip và allocated_chem, vì chúng vẫn biểu diễn liên kết M:N theo mô hình đã duyệt.

grant_laboratory có khóa ghép (grant_id,lab_id), biểu diễn phòng được đề tài sử dụng. chemical_certification có khóa ghép (chemical_id,cert_id), biểu diễn tất cả chứng nhận bắt buộc để xuất một hóa chất. Với High/Extreme, thiếu cấu hình chứng nhận bị từ chối theo nguyên tắc đóng khi chưa đủ chính sách.

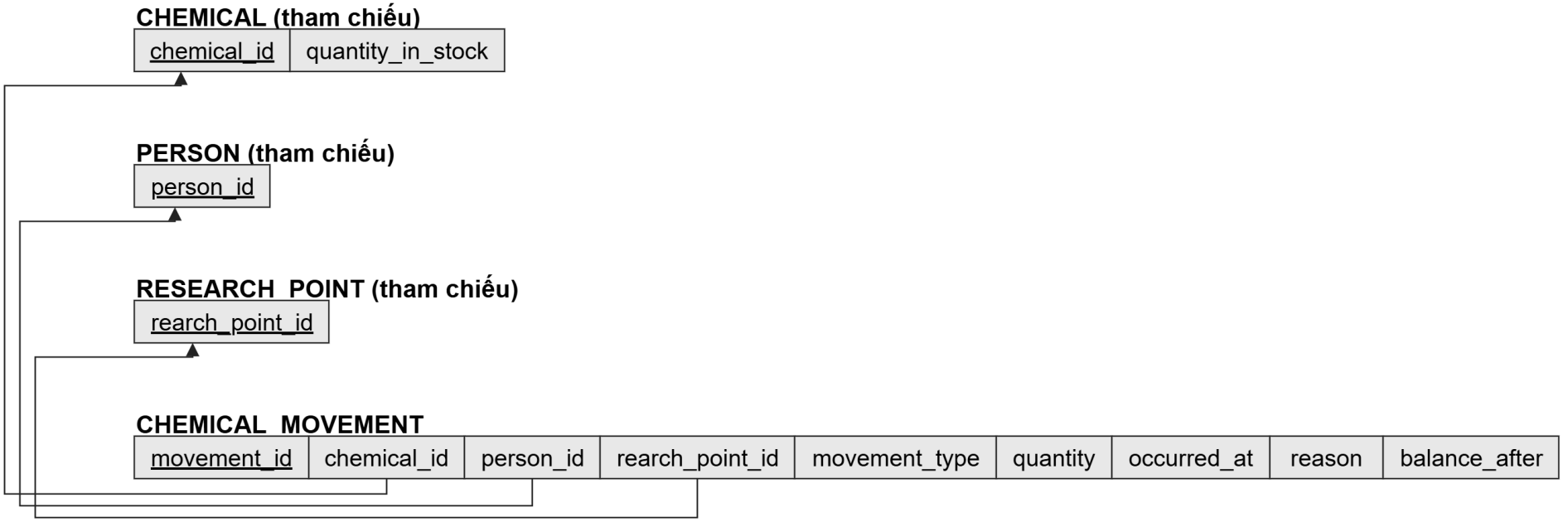
chemical_movement là sổ bất biến các lần nhập Receipt và xuất Issue. quantity_in_stock giữ vai trò số dư hiện hành; mỗi thay đổi số dư phải đi kèm một dòng ledger trong cùng giao dịch. Mọi hóa chất được tạo với số tồn 0 rồi nhập qua receive_chemical. quantity và số dư dùng numeric với ba chữ số thập phân, tách theo đơn vị từng hóa chất.

Các bổ sung vẫn thỏa 3NF theo phụ thuộc hàm đang xét: hai bảng ánh xạ chỉ có khóa ghép; movement_id xác định một sự kiện. balance_after là ảnh chụp kết quả của sự kiện, không thay thế số dư hiện tại. Kiểm toán đối chiếu tổng Receipt trừ tổng Issue với quantity_in_stock để phát hiện sai lệch.

Hình 4.1a Phòng được cấp cho đề tài và chứng nhận bắt buộc của hóa chất



Hình 4.1b Sổ biến động tồn kho hóa chất và các khóa ngoại tham chiếu



Sơ đồ tập trung vào khóa của ba bảng bổ sung. Từ điển dữ liệu của 16 bảng nền được tham chiếu tại Phase 2, mục 3. Phụ lục A của Phase 3 ghi các khác biệt vật lý và từ điển của ba bảng bổ sung; DDL đầy đủ tại mục 4.1.5 là định nghĩa thực thi cuối cùng.

4.1.2 Ràng buộc và trigger

PK, FK, NOT NULL, UNIQUE và CHECK kiểm tra miền giá trị, thứ tự ngày, ngân sách và tồn kho. Những điều kiện liên bảng được xử lý bằng trigger hoặc hàm nghiệp vụ. Bản ghi người và lớp con phải được tạo trong cùng giao dịch; constraint trigger chạy trước commit và yêu cầu đúng một lớp con. Mã định danh người và lớp con không được đổi.

enforce_access kiểm tra trạng thái Active, lớp con, giờ truy cập, người phê duyệt và chứng nhận tại ngày vào. Khung giờ sinh viên là 07:00≤giờ<17:00. Faculty phải có purpose hợp lệ ở mọi thời điểm; Technician được vào 24/7 nhưng vẫn phải có chứng nhận khi vào Restricted/Hazardous. exit_time có thể NULL khi lượt chưa kết thúc.

enforce_borrow lấy khóa người rồi khóa thiết bị trước khi kiểm tra giới hạn ba lượt chưa trả và trạng thái Available. Khi trả, hàm xác định Returned hoặc Overdue và đưa thiết bị về Available. Khóa duy nhất có điều kiện trên equipment_id WHERE actual_return_date IS NULL ngăn cùng thiết bị có hai người giữ.

enforce_allocation kiểm tra đề tài Active và phòng tài sản thuộc grant_laboratory. Phân bổ cũ được giữ để bảo toàn lịch sử sau khi đề tài Completed/Suspended. Khi tài sản còn được phân bổ, phải gỡ liên kết trước khi chuyển phòng. Không đổi grant của hạng mục đang có tài sản.

Trạng thái thời gian không tự thay đổi chỉ nhờ trigger khi không có thao tác ghi. sync_cert_status chuẩn hóa mỗi lần ghi; refresh_temporal_status phải được backend gọi hằng ngày và khi bắt đầu ca làm việc. Các view và các hàm nghiệp vụ vẫn so ngày trực tiếp, vì vậy dữ liệu hết hạn không được chấp nhận dù cột status chưa đến lần làm mới kế tiếp.

4.1.3 Giao dịch và kiểm soát đồng thời

issue_chemical thực hiện trọn một thao tác nguyên tử: xác minh người Active; khóa hạng mục và đề tài; khóa hóa chất; kiểm tra hạn dùng, phân bổ và phòng; khóa các chứng nhận phù hợp; kiểm tra toàn bộ chứng nhận bắt buộc; trừ tồn bằng UPDATE có điều kiện; ghi chemical_movement; trả movement_id. Bất kỳ lỗi nào đều hủy cả thay đổi tồn và ghi sổ.

receive_chemical chỉ chấp nhận Technician Active và số lượng dương tối đa ba chữ số thập phân. Hàm cộng tồn và ghi Receipt trong cùng giao dịch. Không cho sửa hoặc xóa dòng ledger. Vật tư đã quá hạn có thể được tiếp nhận từ dữ liệu kiểm kê nhưng không được xuất sử dụng; quy trình xử lý/hủy kho cần được bổ sung riêng nếu đưa vào vận hành thật.

Ở READ COMMITTED, hai lượt mượn cùng người được tuần tự hóa qua hàng person; hai lượt xuất cùng hóa chất qua hàng chemical. Giao dịch đến sau kiểm tra lại số lượt hoặc tồn sau khi chờ khóa. Nếu dùng REPEATABLE READ/SERIALIZABLE, backend phải xử lý SQLSTATE 40001 bằng cách thử lại toàn giao dịch có giới hạn.

06_concurrency.mjs chuẩn bị tình huống T14: người đang giữ hai thiết bị, hai phiên cùng mượn thêm; chỉ một phiên được hoàn tất. H13 dùng tồn Acetone 8 L, hai phiên cùng xuất 5 L; chỉ một phiên được hoàn tất và tồn cuối 3 L. Tập lệnh còn quan sát wait_event_type=Lock để chứng minh có cạnh tranh khóa thực. Hai kịch bản này chưa được chạy trong phiên PGlite một kết nối.

4.1.4 Dữ liệu mẫu và trình tự thực thi

Bộ dữ liệu có 12 người (3 Faculty, 6 Student, 3 Technician), 3 phòng ở ba mức an toàn, 4 đề tài với Active/Completed/Suspended, 6 hạng mục, 12 thiết bị, 6 hóa chất, 3 loại chứng nhận, 16 dòng sở hữu chứng nhận, 10 phân bổ thiết bị và 7 phân bổ hóa chất. Có chứng nhận đã hết hạn, hóa chất sắp/quá hạn, thiết bị mượn quá hạn, đang bảo trì và đã Retired để kiểm chứng nhánh nghiệp vụ.

Nạp theo 01_schema.sql → 02_seed.sql → 04_rbac.sql; chạy 03_queries.sql để lấy kết quả quản trị và 05_tests.sql để kiểm chứng. Schema dành cho cơ sở dữ liệu mới. Script kiểm thử chạy trong BEGIN/ROLLBACK nên không giữ thay đổi nghiệp vụ của các ca thử; bộ seed giữ nguyên để truy vấn cho kết quả lặp lại.

Trên PostgreSQL máy chủ có thể chạy lần lượt psql -v ON_ERROR_STOP=1 -f cho từng file. Với môi trường kiểm chứng đã cung cấp, cài dependency @electric-sql/pglite rồi chạy node run_validation.mjs. Kiểm thử nhiều phiên dùng dependency pg và DATABASE_URL trỏ đến database thử nghiệm mới, sau đó chạy node 06_concurrency.mjs.

Định danh BIGINT tự sinh được setval sau khi nạp các mã cố định. Những ngày tương đối CURRENT_DATE giúp bộ dữ liệu vẫn kiểm tra được hết hạn và quá hạn ở ngày chạy khác. Múi giờ kết nối và role được đặt Asia/Ho_Chi_Minh; tiền tệ thống nhất VND, không cộng số lượng hóa chất khác đơn vị.

Tập lệnh	Nội dung
01_schema.sql	DDL đầy đủ 19 bảng, FK/CHECK/index, 23 trigger, 5 view, hàm nghiệp vụ và thu hồi quyền PUBLIC.
02_seed.sql	DML mô phỏng, nhập/xuất qua hàm có kiểm soát và reset identity sequence.
03_queries.sql	12 truy vấn quản trị phức hợp.
04_rbac.sql	Tạo bốn role NOLOGIN và GRANT/REVOKE chi tiết.
05_tests.sql	44 xác nhận nghiệp vụ trong giao dịch có rollback.

Tập lệnh	Nội dung
run_validation.mjs	Chạy PGlite, thu kết quả truy vấn, nghiệp vụ, SET ROLE và catalog thực tế.
validation_results.json	Bằng chứng kết quả lần chạy thực tế.
06_concurrency.mjs	Kiểm thử T14 và H13 trên PostgreSQL nhiều phiên; chưa thực thi trong phiên hiện tại.
../completeness_review/run_query_performance.mjs	Tạo PGlite trong bộ nhớ; chạy 12 truy vấn nguyên bản và EXPLAIN ANALYZE, không thay đổi SQL nguồn.
../completeness_review/query_performance_results.json	Kết quả thực đo, mẫu thời gian, kế hoạch thực thi, số hàng và SHA-256 nguồn SQL.
../completeness_review/query_performance_notes.md	Cách tái lập, diễn giải và giới hạn của phép đo.

4.1.5 Mã DDL đầy đủ

Tập lệnh 01_schema.sql tạo schema lab_tracker trong cơ sở dữ liệu mới. Toàn bộ bảng, chỉ mục, view, trigger và hàm nghiệp vụ nằm trong cùng tập lệnh dưới đây.

```
CREATE SCHEMA lab_tracker;

SET search_path = lab_tracker, pg_catalog;

SET TIME ZONE 'Asia/Ho_Chi_Minh';

CREATE TABLE person (
    person_id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    full_name varchar(150) NOT NULL CHECK (btrim(full_name) <> ''),
    email varchar(254) NOT NULL CHECK (email ~
        '^[^[:space:]@]+@[^[:space:]@]+\.[^[:space:]@]+$'),
    phone varchar(20),
    date_of_birth date,
    status varchar(10) NOT NULL DEFAULT 'Active' CHECK (status IN ('Active',
        'Inactive'))
);

CREATE UNIQUE INDEX uq_person_email ON person(lower(email));

CREATE TABLE faculty (
    faculty_id bigint PRIMARY KEY REFERENCES person ON DELETE RESTRICT ON UPDATE
        RESTRICT,
    department varchar(100) NOT NULL,
    academic_rank varchar(100) NOT NULL,
    office_location varchar(100)
);

CREATE TABLE student (
    student_id bigint PRIMARY KEY REFERENCES person ON DELETE RESTRICT ON UPDATE
        RESTRICT,
    student_code varchar(20) NOT NULL UNIQUE,
```

```
    major varchar(100) NOT NULL,
    year_of_study smallint NOT NULL CHECK (year_of_study > 0)
);

CREATE TABLE technician (
    technician_id bigint PRIMARY KEY REFERENCES person ON DELETE RESTRICT ON UPDATE
        RESTRICT,
    employee_code varchar(20) NOT NULL UNIQUE,
    specialty varchar(150) NOT NULL
);

CREATE TABLE laboratory (
    lab_id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    lab_name varchar(150) NOT NULL,
    building varchar(100) NOT NULL,
    room_number varchar(20) NOT NULL,
    capacity integer NOT NULL CHECK (capacity > 0),
    safety_level varchar(12) NOT NULL DEFAULT 'Normal' CHECK (safety_level IN
        ('Normal', 'Restricted', 'Hazardous')),
    UNIQUE(building, room_number)
);

CREATE TABLE research_grant (
    grant_id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    grant_title varchar(200) NOT NULL,
    principal_investigator_id bigint NOT NULL REFERENCES faculty ON DELETE RESTRICT
        ON UPDATE RESTRICT,
    start_date date NOT NULL,
    end_date date NOT NULL,
    total_budget numeric(18, 2) NOT NULL CHECK (total_budget >= 0),
    remaining_budget numeric(18, 2) NOT NULL CHECK (remaining_budget >= 0 AND
        remaining_budget <= total_budget),
    status varchar(12) NOT NULL DEFAULT 'Active' CHECK (status IN ('Active',
        'Completed', 'Suspended')),
    CHECK (end_date > start_date)
);

CREATE TABLE research_point (
    rearch_point_id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    grant_id bigint NOT NULL REFERENCES research_grant ON DELETE RESTRICT ON UPDATE
        RESTRICT,
    point_name varchar(200) NOT NULL,
    description varchar(1000),
    start_date date NOT NULL,
    end_date date NOT NULL,
    status varchar(30) NOT NULL CHECK (status IN ('Planned', 'Active', 'Completed',
        'Suspended')),
    CHECK (end_date > start_date)
);

CREATE TABLE equipment (
    equipment_id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    lab_id bigint NOT NULL REFERENCES laboratory ON DELETE RESTRICT ON UPDATE
        RESTRICT,
    equipment_name varchar(150) NOT NULL,
    serial_number varchar(100) NOT NULL UNIQUE,
    purchase_date date NOT NULL,
    purchase_cost numeric(18, 2) NOT NULL CHECK (purchase_cost >= 0),
```

```
    current_status varchar(20) NOT NULL DEFAULT 'Available' CHECK (current_status IN
        ('Available', 'In Use', 'Under Maintenance', 'Retired'))
);

CREATE TABLE chemical (
    chemical_id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    lab_id bigint NOT NULL REFERENCES laboratory ON DELETE RESTRICT ON UPDATE
        RESTRICT,
    chemical_name varchar(150) NOT NULL,
    cas_number varchar(20) NOT NULL,
    quantity_in_stock numeric(14, 3) NOT NULL DEFAULT 0 CHECK (quantity_in_stock >=
        0),
    unit varchar(10) NOT NULL CHECK (unit IN ('g', 'kg', 'mL', 'L', 'mol')),
    expiry_date date NOT NULL,
    hazard_level varchar(10) NOT NULL DEFAULT 'None' CHECK (hazard_level IN ('None',
        'Low', 'Medium', 'High', 'Extreme')),
    storage_condition varchar(500) NOT NULL
);

CREATE TABLE certification (
    cert_id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    cert_name varchar(150) NOT NULL UNIQUE,
    issuing_authority varchar(150) NOT NULL,
    validity_period_months smallint NOT NULL CHECK (validity_period_months > 0)
);

CREATE TABLE has_certification (
    person_id bigint NOT NULL REFERENCES person ON DELETE RESTRICT ON UPDATE
        RESTRICT,
    cert_id bigint NOT NULL REFERENCES certification ON DELETE RESTRICT ON UPDATE
        RESTRICT,
    issue_date date NOT NULL,
    expiry_date date NOT NULL,
    status varchar(10) NOT NULL DEFAULT 'Valid' CHECK (status IN ('Valid',
        'Expired')),
    PRIMARY KEY(person_id, cert_id),
    CHECK (expiry_date > issue_date)
);

CREATE TABLE allocated_equip (
    rearch_point_id bigint NOT NULL REFERENCES research_point ON DELETE RESTRICT ON
        UPDATE RESTRICT,
    equipment_id bigint NOT NULL REFERENCES equipment ON DELETE RESTRICT ON UPDATE
        RESTRICT,
    PRIMARY KEY(rearch_point_id, equipment_id)
);

CREATE TABLE allocated_chem (
    rearch_point_id bigint NOT NULL REFERENCES research_point ON DELETE RESTRICT ON
        UPDATE RESTRICT,
    chemical_id bigint NOT NULL REFERENCES chemical ON DELETE RESTRICT ON UPDATE
        RESTRICT,
    PRIMARY KEY(rearch_point_id, chemical_id)
);

CREATE TABLE access_log (
    log_id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    person_id bigint NOT NULL REFERENCES person ON DELETE RESTRICT ON UPDATE
```

```
        RESTRICT,
    lab_id bigint NOT NULL REFERENCES laboratory ON DELETE RESTRICT ON UPDATE
        RESTRICT,
    entry_time timestamp NOT NULL,
    exit_time timestamp,
    purpose varchar(500) NOT NULL CHECK (btrim(purpose) <> ''),
    approved_by bigint REFERENCES person ON DELETE RESTRICT ON UPDATE RESTRICT,
    CHECK (exit_time IS NULL OR exit_time > entry_time)
);

CREATE TABLE borrow_transaction (
    borrow_id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    person_id bigint NOT NULL REFERENCES person ON DELETE RESTRICT ON UPDATE
        RESTRICT,
    equipment_id bigint NOT NULL REFERENCES equipment ON DELETE RESTRICT ON UPDATE
        RESTRICT,
    borrow_date date NOT NULL,
    expected_return_date date NOT NULL,
    actual_return_date date,
    status varchar(10) NOT NULL DEFAULT 'Borrowed' CHECK (status IN ('Borrowed',
        'Returned', 'Overdue')),
    CHECK (expected_return_date >= borrow_date),
    CHECK (actual_return_date IS NULL OR actual_return_date >= borrow_date)
);

CREATE UNIQUE INDEX uq_open_equipment_loan ON borrow_transaction(equipment_id)
WHERE actual_return_date IS NULL;

CREATE TABLE maintenance_log (
    maintenance_id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    technician_id bigint NOT NULL REFERENCES technician ON DELETE RESTRICT ON UPDATE
        RESTRICT,
    equipment_id bigint NOT NULL REFERENCES equipment ON DELETE RESTRICT ON UPDATE
        RESTRICT,
    start_date date NOT NULL,
    end_date date,
    description varchar(1000) NOT NULL CHECK (btrim(description) <> ''),
    cost numeric(18, 2) NOT NULL CHECK (cost >= 0),
    CHECK (end_date IS NULL OR end_date >= start_date)
);

CREATE UNIQUE INDEX uq_open_maintenance ON maintenance_log(equipment_id)
WHERE end_date IS NULL;

CREATE TABLE grant_laboratory (
    grant_id bigint NOT NULL REFERENCES research_grant ON DELETE RESTRICT ON UPDATE
        RESTRICT,
    lab_id bigint NOT NULL REFERENCES laboratory ON DELETE RESTRICT ON UPDATE
        RESTRICT,
    PRIMARY KEY(grant_id, lab_id)
);

CREATE TABLE chemical_certification (
    chemical_id bigint NOT NULL REFERENCES chemical ON DELETE RESTRICT ON UPDATE
        RESTRICT,
    cert_id bigint NOT NULL REFERENCES certification ON DELETE RESTRICT ON UPDATE
        RESTRICT,
    PRIMARY KEY(chemical_id, cert_id)
```

```
);

CREATE TABLE chemical_movement (
    movement_id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    chemical_id bigint NOT NULL REFERENCES chemical ON DELETE RESTRICT ON UPDATE
        RESTRICT,
    person_id bigint NOT NULL REFERENCES person ON DELETE RESTRICT ON UPDATE
        RESTRICT,
    rearch_point_id bigint REFERENCES research_point ON DELETE RESTRICT ON UPDATE
        RESTRICT,
    movement_type varchar(10) NOT NULL CHECK (movement_type IN ('Receipt',
        'Issue')),
    quantity numeric(14, 3) NOT NULL CHECK (quantity > 0),
    occurred_at timestamptz NOT NULL DEFAULT clock_timestamp(),
    reason varchar(500) NOT NULL CHECK (btrim(reason) <> ''),
    balance_after numeric(14, 3) NOT NULL CHECK (balance_after >= 0),
    CHECK ( (movement_type = 'Issue' AND rearch_point_id IS NOT NULL) OR
        (movement_type = 'Receipt' AND rearch_point_id IS NULL))
);

CREATE INDEX ix_point_grant ON research_point(grant_id);

CREATE INDEX ix_equipment_lab ON equipment(lab_id);

CREATE INDEX ix_chemical_lab_expiry ON chemical(lab_id,
            expiry_date);

CREATE INDEX ix_cert_person_expiry ON has_certification(person_id,
            expiry_date);

CREATE INDEX ix_access_lab_entry ON access_log(lab_id,
            entry_time DESC);

CREATE INDEX ix_access_person_entry ON access_log(person_id,
            entry_time DESC);

CREATE INDEX ix_borrow_person_open ON borrow_transaction(person_id)
WHERE actual_return_date IS NULL;

CREATE INDEX ix_borrow_equipment_history ON borrow_transaction(equipment_id,
            borrow_date DESC);

CREATE INDEX ix_maintenance_equipment ON maintenance_log(equipment_id,
            start_date DESC);

CREATE INDEX ix_movement_chemical_time ON chemical_movement(chemical_id,
            occurred_at DESC);

CREATE INDEX ix_allocated_equip_equipment ON allocated_equip(equipment_id);

CREATE INDEX ix_allocated_chem_chemical ON allocated_chem(chemical_id);

CREATE FUNCTION enforce_person_subtype()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = lab_tracker, pg_catalog, pg_temp
AS $$
DECLARE
```

```
    v_id bigint;
    v_count integer;
BEGIN
    v_id := coalesce(to_jsonb(NEW), to_jsonb(OLD)) ->> (TG_TABLE_NAME || '_id');
    IF EXISTS (SELECT 1 FROM person WHERE person_id = v_id) THEN
        PERFORM 1 FROM person WHERE person_id = v_id FOR UPDATE;
        SELECT (EXISTS (SELECT 1 FROM faculty WHERE faculty_id = v_id))::integer
        + (EXISTS (SELECT 1 FROM student WHERE student_id = v_id))::integer
        + (EXISTS (SELECT 1 FROM technician WHERE technician_id = v_id))::integer
            INTO v_count;
        IF v_count <> 1 THEN
            RAISE EXCEPTION 'BR01: person % must have exactly one subtype', v_id
                USING ERRCODE = '23514';
        END IF;
    END IF;
    RETURN NULL;
END
$$;

CREATE CONSTRAINT TRIGGER ct_person_subtype
AFTER INSERT OR UPDATE OR DELETE ON person
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW
EXECUTE FUNCTION enforce_person_subtype();

CREATE CONSTRAINT TRIGGER ct_faculty_subtype
AFTER INSERT OR UPDATE OR DELETE ON faculty
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW
EXECUTE FUNCTION enforce_person_subtype();

CREATE CONSTRAINT TRIGGER ct_student_subtype
AFTER INSERT OR UPDATE OR DELETE ON student
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW
EXECUTE FUNCTION enforce_person_subtype();

CREATE CONSTRAINT TRIGGER ct_technician_subtype
AFTER INSERT OR UPDATE OR DELETE ON technician
DEFERRABLE INITIALLY DEFERRED
FOR EACH ROW
EXECUTE FUNCTION enforce_person_subtype();

CREATE FUNCTION immutable_person_key()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = lab_tracker, pg_catalog, pg_temp
AS $$
BEGIN
    IF (to_jsonb(NEW) ->> (TG_TABLE_NAME || '_id')) IS DISTINCT FROM (to_jsonb(OLD)
        ->> (TG_TABLE_NAME || '_id')) THEN
        RAISE EXCEPTION 'BR01: person and subtype identifiers are immutable';
    END IF;
    RETURN NEW;
END
$$;

CREATE TRIGGER tr_person_key
```

```
BEFORE UPDATE ON person
FOR EACH ROW
EXECUTE FUNCTION immutable_person_key();

CREATE TRIGGER tr_faculty_key
BEFORE UPDATE ON faculty
FOR EACH ROW
EXECUTE FUNCTION immutable_person_key();

CREATE TRIGGER tr_student_key
BEFORE UPDATE ON student
FOR EACH ROW
EXECUTE FUNCTION immutable_person_key();

CREATE TRIGGER tr_technician_key
BEFORE UPDATE ON technician
FOR EACH ROW
EXECUTE FUNCTION immutable_person_key();

CREATE FUNCTION sync_cert_status()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = lab_tracker, pg_catalog, pg_temp
AS $$
BEGIN
    NEW.status := CASE WHEN NEW.expiry_date < CURRENT_DATE THEN 'Expired' ELSE
        'Valid' END;
    RETURN NEW;
END
$$;

CREATE TRIGGER tr_cert_status
BEFORE INSERT OR UPDATE ON has_certification
FOR EACH ROW
EXECUTE FUNCTION sync_cert_status();

CREATE FUNCTION refresh_temporal_status()
RETURNS integer
LANGUAGE plpgsql SECURITY DEFINER
SET search_path = lab_tracker, pg_catalog, pg_temp
AS $$
DECLARE
    v_count integer;
BEGIN
    UPDATE has_certification SET status = 'Expired' WHERE expiry_date < CURRENT_DATE
        AND status <> 'Expired';
    GET DIAGNOSTICS v_count = ROW_COUNT;
    UPDATE borrow_transaction SET status = 'Overdue' WHERE actual_return_date IS
        NULL AND expected_return_date < CURRENT_DATE AND status <> 'Overdue';
    RETURN v_count;
END
$$;

CREATE FUNCTION enforce_access()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = lab_tracker, pg_catalog, pg_temp
AS $$
```

```
DECLARE
    v_level text;
    v_day date;
BEGIN
    IF TG_OP = 'UPDATE' AND (NEW.person_id, NEW.lab_id, NEW.entry_time, NEW.purpose,
        NEW.approved_by) IS DISTINCT FROM (OLD.person_id, OLD.lab_id,
        OLD.entry_time, OLD.purpose, OLD.approved_by) THEN
        RAISE EXCEPTION 'BR21: access identity and approval are immutable';
    END IF;
    IF TG_OP = 'UPDATE' THEN
        RETURN NEW;
    END IF;
    PERFORM 1 FROM person WHERE person_id = NEW.person_id AND status = 'Active' FOR
        SHARE;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'BR23: inactive or missing person';
    END IF;
    SELECT safety_level INTO v_level FROM laboratory WHERE lab_id = NEW.lab_id FOR
        SHARE;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'BR25: laboratory not found';
    END IF;
    v_day := NEW.entry_time::date;
    IF NEW.approved_by IS NOT NULL THEN
        PERFORM 1 FROM person p WHERE p.person_id = NEW.approved_by AND p.status =
            'Active'
        AND (EXISTS (SELECT 1 FROM faculty WHERE faculty_id = p.person_id) OR EXISTS
            (SELECT 1 FROM technician WHERE technician_id = p.person_id)) FOR SHARE;
        IF NOT FOUND OR NEW.approved_by = NEW.person_id THEN
            RAISE EXCEPTION
                'BR04: approval must be from another active faculty or technician';
        END IF;
    END IF;
    IF EXISTS (SELECT 1 FROM student WHERE student_id = NEW.person_id)
    AND (NEW.entry_time::time < time '07:00' OR NEW.entry_time::time >= time
        '17:00') AND NEW.approved_by IS NULL THEN
        RAISE EXCEPTION 'BR04: student out-of-hours approval required';
    END IF;
    IF v_level IN ('Restricted', 'Hazardous') THEN
        PERFORM 1 FROM has_certification WHERE person_id = NEW.person_id FOR SHARE;
    END IF;
    IF v_level IN ('Restricted', 'Hazardous') AND NOT EXISTS (
    SELECT 1 FROM has_certification WHERE person_id = NEW.person_id AND status =
        'Valid' AND issue_date <= v_day AND expiry_date >= v_day
    ) THEN
        RAISE EXCEPTION 'BR05: valid safety certification required';
    END IF;
    RETURN NEW;
END
$$;

CREATE TRIGGER tr_access
BEFORE INSERT OR UPDATE ON access_log
FOR EACH ROW
EXECUTE FUNCTION enforce_access();

CREATE FUNCTION enforce_borrow()
RETURNS trigger
```

```
LANGUAGE plpgsql
SET search_path = lab_tracker, pg_catalog, pg_temp
AS $$
DECLARE
    v_state text;
    v_count integer;
BEGIN
    IF TG_OP = 'UPDATE' AND (NEW.person_id, NEW.equipment_id, NEW.borrow_date,
        NEW.expected_return_date) IS DISTINCT FROM (OLD.person_id, OLD.equipment_id,
        OLD.borrow_date, OLD.expected_return_date) THEN
        RAISE EXCEPTION 'BR21: loan identity and original due date are immutable';
    END IF;
    IF TG_OP = 'UPDATE' AND OLD.actual_return_date IS NOT NULL AND
        NEW.actual_return_date IS DISTINCT FROM OLD.actual_return_date THEN
        RAISE EXCEPTION 'BR21: completed loan is immutable';
    END IF;
    UPDATE person SET status = status WHERE person_id = NEW.person_id AND status =
        'Active';
    IF NOT FOUND AND TG_OP = 'INSERT' THEN
        RAISE EXCEPTION 'BR14: active borrower required';
    END IF;
    SELECT current_status INTO v_state FROM equipment WHERE equipment_id =
        NEW.equipment_id FOR UPDATE;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'BR25: equipment not found';
    END IF;
    IF TG_OP = 'INSERT' AND NEW.actual_return_date IS NULL THEN
        IF v_state <> 'Available' THEN
            RAISE EXCEPTION 'BR13: equipment unavailable: %', v_state;
        END IF;
        SELECT count(*) INTO v_count FROM borrow_transaction WHERE person_id =
            NEW.person_id AND actual_return_date IS NULL;
        IF v_count >= 3 THEN
            RAISE EXCEPTION 'BR14: maximum three open loans';
        END IF;
        UPDATE equipment SET current_status = 'In Use' WHERE equipment_id =
            NEW.equipment_id;
    END IF;
    IF TG_OP = 'UPDATE' AND OLD.actual_return_date IS NULL AND
        NEW.actual_return_date IS NOT NULL THEN
        UPDATE equipment SET current_status = 'Available' WHERE equipment_id =
            NEW.equipment_id AND current_status = 'In Use';
    END IF;
    NEW.status := CASE WHEN NEW.actual_return_date > NEW.expected_return_date OR
        (NEW.actual_return_date IS NULL AND NEW.expected_return_date < CURRENT_DATE)
        THEN 'Overdue'
    WHEN NEW.actual_return_date IS NULL THEN 'Borrowed' ELSE 'Returned' END;
    RETURN NEW;
END
$$;

CREATE TRIGGER tr_borrow
BEFORE INSERT OR UPDATE ON borrow_transaction
FOR EACH ROW
EXECUTE FUNCTION enforce_borrow();

CREATE FUNCTION enforce_asset_and_lab()
RETURNS trigger
```

```
LANGUAGE plpgsql
SET search_path = lab_tracker, pg_catalog, pg_temp
AS $$
DECLARE
    v_level text;
BEGIN
    IF TG_TABLE_NAME = 'chemical' THEN
        SELECT safety_level INTO v_level FROM laboratory WHERE lab_id = NEW.lab_id
            FOR SHARE;
        IF NEW.hazard_level = 'Extreme' AND v_level IS DISTINCT FROM 'Hazardous'
            THEN
            RAISE EXCEPTION 'BR20: Extreme requires Hazardous lab';
        END IF;
        IF TG_OP = 'INSERT' AND NEW.quantity_in_stock <> 0 THEN
            RAISE EXCEPTION 'BR19: initial stock must use receive_chemical';
        END IF;
        IF TG_OP = 'UPDATE' AND NEW.lab_id <> OLD.lab_id AND EXISTS (SELECT 1 FROM
            allocated_chem WHERE chemical_id = NEW.chemical_id) THEN
            RAISE EXCEPTION
                'BR11: remove chemical allocations before lab relocation';
        END IF;
    ELSIF TG_TABLE_NAME = 'equipment' THEN
        IF TG_OP = 'UPDATE' AND NEW.lab_id <> OLD.lab_id AND EXISTS (SELECT 1 FROM
            allocated_equip WHERE equipment_id = NEW.equipment_id) THEN
            RAISE EXCEPTION
                'BR11: remove equipment allocations before lab relocation';
        END IF;
        IF TG_OP = 'UPDATE' AND OLD.current_status = 'In Use' AND NEW.current_status
            IN ('Under Maintenance', 'Retired') AND EXISTS (SELECT 1 FROM
            borrow_transaction WHERE equipment_id = NEW.equipment_id AND
            actual_return_date IS NULL) THEN
            RAISE EXCEPTION 'BR13: cannot retire or maintain borrowed equipment';
        END IF;
    ELSIF NEW.safety_level <> 'Hazardous' AND EXISTS (SELECT 1 FROM chemical WHERE
        lab_id = NEW.lab_id AND hazard_level = 'Extreme') THEN
        RAISE EXCEPTION 'BR20: lab contains Extreme chemical';
    END IF;
    RETURN NEW;
END
$$;

CREATE TRIGGER tr_chemical_storage
BEFORE INSERT OR UPDATE ON chemical
FOR EACH ROW
EXECUTE FUNCTION enforce_asset_and_lab();

CREATE TRIGGER tr_equipment_lab
BEFORE INSERT OR UPDATE ON equipment
FOR EACH ROW
EXECUTE FUNCTION enforce_asset_and_lab();

CREATE TRIGGER tr_lab_level
BEFORE UPDATE ON laboratory
FOR EACH ROW
EXECUTE FUNCTION enforce_asset_and_lab();

CREATE FUNCTION enforce_allocation()
RETURNS trigger
```

```
LANGUAGE plpgsql
SET search_path = lab_tracker, pg_catalog, pg_temp
AS $$
DECLARE
    v_grant bigint;
    v_lab bigint;
    v_status text;
BEGIN
    SELECT p.grant_id, g.status INTO v_grant, v_status FROM research_point p JOIN
        research_grant g USING(grant_id) WHERE p.rearch_point_id =
        NEW.rearch_point_id FOR SHARE OF g, p;
    IF NOT FOUND OR v_status <> 'Active' THEN
        RAISE EXCEPTION 'BR12: allocation requires Active grant';
    END IF;
    IF TG_TABLE_NAME = 'allocated_equip' THEN
        SELECT lab_id INTO v_lab FROM equipment WHERE equipment_id =
            NEW.equipment_id FOR SHARE;
    ELSE
        SELECT lab_id INTO v_lab FROM chemical WHERE chemical_id = NEW.chemical_id
            FOR SHARE;
    END IF;
    PERFORM 1 FROM grant_laboratory WHERE grant_id = v_grant AND lab_id = v_lab FOR
        SHARE;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'BR11: asset lab is not assigned to grant';
    END IF;
    RETURN NEW;
END
$$;

CREATE TRIGGER tr_allocate_equip
BEFORE INSERT OR UPDATE ON allocated_equip
FOR EACH ROW
EXECUTE FUNCTION enforce_allocation();

CREATE TRIGGER tr_allocate_chem
BEFORE INSERT OR UPDATE ON allocated_chem
FOR EACH ROW
EXECUTE FUNCTION enforce_allocation();

CREATE FUNCTION protect_grant_lab()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = lab_tracker, pg_catalog, pg_temp
AS $$
BEGIN
    PERFORM 1 FROM research_grant WHERE grant_id = OLD.grant_id FOR UPDATE;
    IF EXISTS (SELECT 1 FROM allocated_equip a JOIN research_point p
        USING(rearch_point_id) JOIN equipment e USING(equipment_id) WHERE p.grant_id
        = OLD.grant_id AND e.lab_id = OLD.lab_id)
    OR EXISTS (SELECT 1 FROM allocated_chem a JOIN research_point p
        USING(rearch_point_id) JOIN chemical c USING(chemical_id) WHERE p.grant_id =
        OLD.grant_id AND c.lab_id = OLD.lab_id) THEN
        RAISE EXCEPTION 'BR11: remove allocations before grant lab membership';
    END IF;
    IF TG_OP = 'UPDATE' THEN
        RETURN NEW;
    ELSE
```

```
        RETURN OLD;
    END IF;
END
$$;

CREATE TRIGGER tr_protect_grant_lab
BEFORE DELETE OR UPDATE ON grant_laboratory
FOR EACH ROW
EXECUTE FUNCTION protect_grant_lab();

CREATE FUNCTION protect_point_grant()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = lab_tracker, pg_catalog, pg_temp
AS $$
BEGIN
    IF NEW.grant_id <> OLD.grant_id AND (EXISTS (SELECT 1 FROM allocated_equip WHERE
        rearch_point_id = OLD.rearch_point_id) OR EXISTS (SELECT 1 FROM
        allocated_chem WHERE rearch_point_id = OLD.rearch_point_id)) THEN
        RAISE EXCEPTION 'BR11: allocated research point cannot change grant';
    END IF;
    RETURN NEW;
END
$$;

CREATE TRIGGER tr_point_grant
BEFORE UPDATE ON research_point
FOR EACH ROW
EXECUTE FUNCTION protect_point_grant();

CREATE FUNCTION enforce_maintenance()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = lab_tracker, pg_catalog, pg_temp
AS $$
DECLARE
    v_state text;
BEGIN
    IF TG_OP = 'UPDATE' AND (NEW.technician_id, NEW.equipment_id, NEW.start_date) IS
        DISTINCT FROM (OLD.technician_id, OLD.equipment_id, OLD.start_date) THEN
        RAISE EXCEPTION 'BR03: maintenance identity is immutable';
    END IF;
    SELECT current_status INTO v_state FROM equipment WHERE equipment_id =
        NEW.equipment_id FOR UPDATE;
    IF NEW.end_date IS NULL THEN
        IF EXISTS (SELECT 1 FROM borrow_transaction WHERE equipment_id =
            NEW.equipment_id AND actual_return_date IS NULL) OR v_state = 'Retired'
            THEN
            RAISE EXCEPTION 'BR13: equipment cannot enter maintenance';
        END IF;
        UPDATE equipment SET current_status = 'Under Maintenance' WHERE equipment_id
            = NEW.equipment_id;
    ELSIF TG_OP = 'UPDATE' AND OLD.end_date IS NULL THEN
        UPDATE equipment SET current_status = 'Available' WHERE equipment_id =
            NEW.equipment_id;
    END IF;
    RETURN NEW;
END
```

```
$$;

CREATE TRIGGER tr_maintenance
BEFORE INSERT OR UPDATE ON maintenance_log
FOR EACH ROW
EXECUTE FUNCTION enforce_maintenance();

CREATE FUNCTION protect_history()
RETURNS trigger
LANGUAGE plpgsql
SET search_path = lab_tracker, pg_catalog, pg_temp
AS $$
BEGIN
    RAISE EXCEPTION 'BR19/21: history is append-only; use controlled workflow';
END
$$;

CREATE TRIGGER tr_movement_history
BEFORE UPDATE OR DELETE ON chemical_movement
FOR EACH ROW
EXECUTE FUNCTION protect_history();

CREATE TRIGGER tr_loan_delete
BEFORE DELETE ON borrow_transaction
FOR EACH ROW
EXECUTE FUNCTION protect_history();

CREATE TRIGGER tr_access_delete
BEFORE DELETE ON access_log
FOR EACH ROW
EXECUTE FUNCTION protect_history();

CREATE TRIGGER tr_maintenance_delete
BEFORE DELETE ON maintenance_log
FOR EACH ROW
EXECUTE FUNCTION protect_history();

CREATE FUNCTION receive_chemical(p_chemical bigint, p_actor bigint, p_quantity
    numeric, p_reason text)
RETURNS bigint
LANGUAGE plpgsql SECURITY DEFINER
SET search_path = lab_tracker, pg_catalog, pg_temp
AS $$
DECLARE
    v_balance numeric;
    v_id bigint;
BEGIN
    IF p_quantity IS NULL OR p_quantity <= 0 OR p_quantity <> round(p_quantity, 3)
        THEN
        RAISE EXCEPTION 'BR19: positive quantity with at most 3 decimals required';
    END IF;
    PERFORM 1 FROM person p JOIN technician t ON t.technician_id = p.person_id WHERE
        p.person_id = p_actor AND p.status = 'Active' FOR SHARE OF p;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'BR19: active technician required';
    END IF;
    UPDATE chemical SET quantity_in_stock = quantity_in_stock + p_quantity WHERE
        chemical_id = p_chemical RETURNING quantity_in_stock INTO v_balance;
```

```
    IF NOT FOUND THEN
        RAISE EXCEPTION 'BR25: chemical not found';
    END IF;
    INSERT INTO chemical_movement(chemical_id, person_id, movement_type, quantity,
        reason, balance_after)
    VALUES (p_chemical, p_actor, 'Receipt', p_quantity, p_reason, v_balance)
        RETURNING movement_id INTO v_id;
    RETURN v_id;
END
$$;

CREATE FUNCTION issue_chemical(p_chemical bigint, p_actor bigint, p_point bigint,
    p_quantity numeric, p_reason text)
RETURNS bigint
LANGUAGE plpgsql SECURITY DEFINER
SET search_path = lab_tracker, pg_catalog, pg_temp
AS $$
DECLARE
    v_chemical chemical % ROWTYPE;
    v_id bigint;
    v_grant bigint;
    v_status text;
BEGIN
    IF p_quantity IS NULL OR p_quantity <= 0 OR p_quantity <> round(p_quantity, 3)
        THEN
        RAISE EXCEPTION 'BR19: positive quantity with at most 3 decimals required';
    END IF;
    PERFORM 1 FROM person WHERE person_id = p_actor AND status = 'Active' FOR SHARE;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'BR06: active person required';
    END IF;
    SELECT p.grant_id, g.status INTO v_grant, v_status FROM research_point p JOIN
        research_grant g USING(grant_id) WHERE p.rearch_point_id = p_point FOR SHARE
        OF p, g;
    IF NOT FOUND OR v_status <> 'Active' THEN
        RAISE EXCEPTION 'BR12: issue requires Active grant';
    END IF;
    SELECT * INTO v_chemical FROM chemical WHERE chemical_id = p_chemical FOR
        UPDATE;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'BR25: chemical not found';
    END IF;
    IF v_chemical.expiry_date < CURRENT_DATE THEN
        RAISE EXCEPTION 'BR18: expired chemical cannot be issued';
    END IF;
    PERFORM 1 FROM allocated_chem WHERE rearch_point_id = p_point AND chemical_id =
        p_chemical FOR SHARE;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'BR11: chemical must be allocated to research point';
    END IF;
    PERFORM 1 FROM grant_laboratory WHERE grant_id = v_grant AND lab_id =
        v_chemical.lab_id FOR SHARE;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'BR11: chemical lab is not assigned to grant';
    END IF;
    IF v_chemical.hazard_level IN ('High', 'Extreme') THEN
        PERFORM 1 FROM chemical_certification WHERE chemical_id = p_chemical FOR
            SHARE;
```

```
        IF NOT FOUND THEN
            RAISE EXCEPTION 'BR06: matching certificate policy missing';
        END IF;
        PERFORM 1 FROM has_certification h JOIN chemical_certification r
            USING(cert_id) WHERE r.chemical_id = p_chemical AND h.person_id =
            p_actor FOR SHARE OF h;
        IF EXISTS (SELECT 1 FROM chemical_certification r WHERE r.chemical_id =
            p_chemical AND NOT EXISTS (
        SELECT 1 FROM has_certification h WHERE h.person_id = p_actor AND h.cert_id
            = r.cert_id AND h.status = 'Valid' AND h.issue_date <= CURRENT_DATE AND
            h.expiry_date >= CURRENT_DATE
        )) THEN
            RAISE EXCEPTION 'BR06: all matching valid certificates required';
        END IF;
    END IF;
    UPDATE chemical SET quantity_in_stock = quantity_in_stock - p_quantity WHERE
        chemical_id = p_chemical AND quantity_in_stock >= p_quantity RETURNING
        quantity_in_stock INTO v_chemical.quantity_in_stock;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'BR17: insufficient chemical stock';
    END IF;
    INSERT INTO chemical_movement(chemical_id, person_id, rearch_point_id,
        movement_type, quantity, reason, balance_after)
    VALUES (p_chemical, p_actor, p_point, 'Issue', p_quantity, p_reason,
        v_chemical.quantity_in_stock) RETURNING movement_id INTO v_id;
    RETURN v_id;
END
$$;

CREATE FUNCTION checkout_equipment(p_actor bigint, p_equipment bigint, p_due date)
RETURNS bigint
LANGUAGE plpgsql SECURITY DEFINER
SET search_path = lab_tracker, pg_catalog, pg_temp
AS $$
DECLARE
    v_id bigint;
BEGIN
    INSERT INTO borrow_transaction(person_id, equipment_id, borrow_date,
        expected_return_date)
    VALUES (p_actor, p_equipment, CURRENT_DATE, p_due) RETURNING borrow_id INTO
        v_id;
    RETURN v_id;
END
$$;

CREATE FUNCTION return_equipment(p_actor bigint, p_borrow bigint)
RETURNS void
LANGUAGE plpgsql SECURITY DEFINER
SET search_path = lab_tracker, pg_catalog, pg_temp
AS $$
BEGIN
    UPDATE borrow_transaction SET actual_return_date = CURRENT_DATE WHERE borrow_id
        = p_borrow AND person_id = p_actor AND actual_return_date IS NULL;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Loan not found or already returned';
    END IF;
END
$$;
```

```
CREATE FUNCTION record_access(p_actor bigint, p_lab bigint, p_purpose text,
    p_approver bigint DEFAULT NULL)
RETURNS bigint
LANGUAGE plpgsql SECURITY DEFINER
SET search_path = lab_tracker, pg_catalog, pg_temp
AS $$
DECLARE
    v_id bigint;
BEGIN
    INSERT INTO access_log(person_id, lab_id, entry_time, purpose, approved_by)
    VALUES (p_actor, p_lab, clock_timestamp() AT TIME ZONE 'Asia/Ho_Chi_Minh',
        p_purpose, p_approver) RETURNING log_id INTO v_id;
    RETURN v_id;
END
$$;

CREATE FUNCTION close_access(p_actor bigint, p_log bigint)
RETURNS void
LANGUAGE plpgsql SECURITY DEFINER
SET search_path = lab_tracker, pg_catalog, pg_temp
AS $$
BEGIN
    UPDATE access_log SET exit_time = clock_timestamp() AT TIME ZONE
        'Asia/Ho_Chi_Minh' WHERE log_id = p_log AND person_id = p_actor AND
        exit_time IS NULL;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Open access log not found';
    END IF;
END
$$;

CREATE FUNCTION start_maintenance(p_actor bigint, p_equipment bigint, p_description
    text, p_cost numeric)
RETURNS bigint
LANGUAGE plpgsql SECURITY DEFINER
SET search_path = lab_tracker, pg_catalog, pg_temp
AS $$
DECLARE
    v_id bigint;
BEGIN
    PERFORM 1 FROM person p JOIN technician t ON t.technician_id = p.person_id WHERE
        p.person_id = p_actor AND p.status = 'Active' FOR SHARE OF p;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'BR03: active technician required';
    END IF;
    INSERT INTO maintenance_log(technician_id, equipment_id, start_date,
        description, cost) VALUES (p_actor, p_equipment, CURRENT_DATE,
        p_description, p_cost) RETURNING maintenance_id INTO v_id;
    RETURN v_id;
END
$$;

CREATE FUNCTION finish_maintenance(p_actor bigint, p_maintenance bigint,
    p_description text, p_cost numeric)
RETURNS void
LANGUAGE plpgsql SECURITY DEFINER
SET search_path = lab_tracker, pg_catalog, pg_temp
```

```
AS $$
BEGIN
    PERFORM 1 FROM person p JOIN technician t ON t.technician_id = p.person_id WHERE
        p.person_id = p_actor AND p.status = 'Active' FOR SHARE OF p;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'BR03: active technician required';
    END IF;
    UPDATE maintenance_log SET end_date = CURRENT_DATE, description = p_description,
        cost = p_cost WHERE maintenance_id = p_maintenance AND technician_id =
        p_actor AND end_date IS NULL;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Open maintenance not found for technician';
    END IF;
END
$$;

CREATE VIEW v_person_role AS
SELECT p.person_id,p.full_name,p.email,p.status,
       CASE WHEN f.faculty_id IS NOT NULL THEN 'Faculty' WHEN s.student_id IS NOT
           NULL THEN 'Student' ELSE 'Technician'
       END AS person_role
FROM person p
LEFT JOIN faculty f ON f.faculty_id = p.person_id
LEFT JOIN student s ON s.student_id = p.person_id;

CREATE VIEW v_certification_current AS
SELECT h.person_id,p.full_name,h.cert_id,c.cert_name,h.issue_date,h.expiry_date,
       CASE WHEN h.expiry_date < CURRENT_DATE THEN 'Expired' ELSE h.status
       END AS effective_status
FROM has_certification h
JOIN person p USING(person_id)
JOIN certification c USING(cert_id);

CREATE VIEW v_open_loans AS
SELECT b.borrow_id,b.person_id,p.full_name,b.equipment_id,e.equipment_name,
       e.lab_id,b.borrow_date,b.expected_return_date,
       greatest(CURRENT_DATE - b.expected_return_date, 0) AS days_overdue,
       CASE WHEN b.expected_return_date < CURRENT_DATE THEN 'Overdue' ELSE
           'Borrowed'
       END AS effective_status
FROM borrow_transaction b
JOIN person p USING(person_id)
JOIN equipment e USING(equipment_id)
WHERE b.actual_return_date IS NULL;

CREATE VIEW v_grant_asset_summary AS
SELECT g.grant_id,g.grant_title,g.status,g.total_budget,g.remaining_budget,

    (SELECT count(DISTINCT a.equipment_id)
     FROM research_point p
     JOIN allocated_equip a USING(rearch_point_id)
     WHERE p.grant_id = g.grant_id) AS equipment_count,

    (SELECT count(DISTINCT a.chemical_id)
     FROM research_point p
     JOIN allocated_chem a USING(rearch_point_id)
     WHERE p.grant_id = g.grant_id) AS chemical_count
FROM research_grant g;
```

```sql
CREATE VIEW v_chemical_stock_audit AS
SELECT c.chemical_id,c.chemical_name,c.quantity_in_stock,
       coalesce(sum(CASE
            WHEN m.movement_type = 'Receipt' THEN m.quantity ELSE - m.quantity
                    END), 0)::numeric(14,
            3) AS ledger_balance,
       c.quantity_in_stock = coalesce(sum(CASE
            WHEN m.movement_type = 'Receipt' THEN m.quantity ELSE - m.quantity
            END),
            0) AS reconciled
FROM chemical c
LEFT JOIN chemical_movement m USING(chemical_id)
GROUP BY c.chemical_id;

REVOKE ALL ON SCHEMA lab_tracker FROM PUBLIC;

REVOKE ALL ON ALL TABLES IN SCHEMA lab_tracker FROM PUBLIC;

REVOKE ALL ON ALL SEQUENCES IN SCHEMA lab_tracker FROM PUBLIC;

REVOKE EXECUTE ON ALL FUNCTIONS IN SCHEMA lab_tracker FROM PUBLIC;

ALTER DEFAULT PRIVILEGES IN SCHEMA lab_tracker REVOKE EXECUTE ON FUNCTIONS
FROM PUBLIC;
```

4.1.6 Mã DML tạo dữ liệu mẫu

```sql
SET search_path = lab_tracker, pg_catalog;

SET TIME ZONE 'Asia/Ho_Chi_Minh';

BEGIN;

INSERT INTO person(person_id,
                   full_name, email, phone, date_of_birth, status)
VALUES (1,'Nguyễn Minh An','an.nguyen@university.edu.vn','0901000001','1980-04-15',
    'Active'),
       (2,'Trần Thu Bình','binh.tran@university.edu.vn','0901000002','1985-09-20',
           'Active'),
       (3,'Lê Quang Chi','chi.le@university.edu.vn','0901000003','1978-01-05',
           'Active'),
       (4,'Phạm Hải Dương','duong.pham@university.edu.vn','0901000004','2005-06-12',
           'Active'),
       (5,'Võ Ngọc Hân','han.vo@university.edu.vn','0901000005','2006-11-08',
           'Active'),
       (6,'Đặng Đức Khoa','khoa.dang@university.edu.vn','0901000006','2004-03-17',
           'Active'),
       (7,'Bùi Thanh Lam','lam.bui@university.edu.vn','0901000007','2005-07-21',
           'Active'),
       (8,'Đỗ Anh Minh','minh.do@university.edu.vn','0901000008','2006-05-10',
           'Inactive'),
       (9,'Nguyễn Hoàng Nam','nam.nguyen@university.edu.vn','0901000009',
           '1990-12-25','Active'),
       (10,'Trần Bảo Ngọc','ngoc.tran@university.edu.vn','0901000010','1993-10-03',
           'Active'),
       (11,'Lê Minh Phúc','phuc.le@university.edu.vn','0901000011','1995-08-18',
```

```
            'Active'),
       (12,'Phan Thảo Vy','vy.phan@university.edu.vn','0901000012','2005-02-28',
            'Active');

INSERT INTO faculty
VALUES (1,'Công nghệ thông tin','Tiến sĩ','A-201'),
       (2,'Hóa học','Phó giáo sư','B-301'),
       (3,'Điện tử','Tiến sĩ','C-401');

INSERT INTO student
VALUES (4,'N24DCCN101','Công nghệ thông tin',3),
       (5,'N24DCCN102','Hóa học',2),
       (6,'N23DCCN103','Điện tử',4),
       (7,'N24DCCN104','Công nghệ thông tin',3),
       (8,'N25DCCN105','Hóa học',2),
       (12,'N24DCCN106','Hóa học',3);

INSERT INTO technician
VALUES (9,'KT001','Thiết bị phân tích'),
       (10,'KT002','An toàn hóa chất'),
       (11,'KT003','Điện tử và hiệu chuẩn');

COMMIT;

INSERT INTO laboratory
VALUES (1,'Phòng máy tính nghiên cứu','A','101',30,'Normal'),
       (2,'Phòng phân tích vật liệu','B','201',15,'Restricted'),
       (3,'Phòng phản ứng hóa học','B','202',8,'Hazardous');

INSERT INTO research_grant
VALUES (1,'Phân tích vật liệu cảm biến',2,CURRENT_DATE-180,CURRENT_DATE + 180,
    500000000,320000000,'Active'),
       (2,'Hệ thống giám sát phòng thí nghiệm',1,CURRENT_DATE-90,CURRENT_DATE + 270,
            250000000,190000000,'Active'),
       (3,'Mẫu thử điện tử công suất',3,CURRENT_DATE-400,CURRENT_DATE-30,300000000,
            20000000,'Completed'),
       (4,'Vật liệu lưu trữ năng lượng',2,CURRENT_DATE-60,CURRENT_DATE + 300,
            450000000,410000000,'Suspended');

INSERT INTO research_point
VALUES (1,1,'Tổng hợp mẫu','Tạo vật liệu cảm biến',CURRENT_DATE-150,CURRENT_DATE +
    60,'Active'),
       (2,1,'Đo và phân tích','Đánh giá đặc tính mẫu',CURRENT_DATE-30,CURRENT_DATE +
            150,'Active'),
       (3,2,'Thu thập dữ liệu','Kết nối cảm biến phòng',CURRENT_DATE-60,CURRENT_DATE
            + 90,'Active'),
       (4,2,'Phân tích dữ liệu','Xây dựng chỉ số vận hành',CURRENT_DATE + 1,
            CURRENT_DATE + 240,'Planned'),
       (5,3,'Đo kiểm mẫu thử','Đã hoàn tất nghiệm thu',CURRENT_DATE-350,
            CURRENT_DATE-60,'Completed'),
       (6,4,'Khảo sát vật liệu','Tạm dừng theo quyết định đề tài',CURRENT_DATE-45,
            CURRENT_DATE + 200,'Suspended');

INSERT INTO grant_laboratory
VALUES(1,2),
      (1,3),
      (2,1),
      (2,2),
```

```
      (3,1),
      (4,3);

INSERT INTO equipment
VALUES (1,1,'Máy trạm nghiên cứu 01','WS-2025-001',CURRENT_DATE-500,42000000,
    'Available'),
       (2,1,'Máy trạm nghiên cứu 02','WS-2025-002',CURRENT_DATE-500,42000000,
           'Available'),
       (3,1,'Bộ cảm biến môi trường','IOT-2026-001',CURRENT_DATE-120,12000000,
           'Available'),
       (4,2,'Kính hiển vi quang học','MIC-2024-001',CURRENT_DATE-700,95000000,
           'Available'),
       (5,2,'Cân phân tích 01','BAL-2024-001',CURRENT_DATE-650,35000000,
           'Available'),
       (6,2,'Máy đo phổ UV-Vis','UV-2023-001',CURRENT_DATE-900,220000000,
           'Available'),
       (7,3,'Máy khuấy từ gia nhiệt','STIR-2025-001',CURRENT_DATE-400,18000000,
           'Available'),
       (8,3,'Tủ hút hóa chất','HOOD-2024-001',CURRENT_DATE-600,160000000,
           'Available'),
       (9,1,'Máy hiện sóng cũ','OSC-2018-001',CURRENT_DATE-2500,25000000,'Retired'),
       (10,2,'Cân phân tích 02','BAL-2025-002',CURRENT_DATE-300,36000000,
           'Available'),
       (11,1,'Bộ thu thập dữ liệu','DAQ-2026-001',CURRENT_DATE-100,28000000,
           'Available'),
       (12,1,'Máy tính bảng khảo sát','TAB-2026-001',CURRENT_DATE-80,15000000,
           'Available');

INSERT INTO chemical
VALUES (1,3,'Ethanol 96%','64-17-5',0,'L',CURRENT_DATE + 240,'High',
    'Tủ chống cháy, tránh nguồn nhiệt'),
       (2,3,'Axit nitric 65%','7697-37-2',0,'L',CURRENT_DATE + 180,'Extreme',
           'Tủ axit thông gió, tách chất hữu cơ'),
       (3,2,'Natri clorua','7647-14-5',0,'kg',CURRENT_DATE + 500,'Low',
           'Đậy kín, nơi khô ráo'),
       (4,2,'Đồng sulfat','7758-98-7',0,'g',CURRENT_DATE + 25,'Medium',
           'Bình kín, tránh ẩm'),
       (5,3,'Methanol','67-56-1',0,'L',CURRENT_DATE-5,'High',
           'Tủ chống cháy, khu cách ly hết hạn'),
       (6,3,'Acetone','67-64-1',0,'L',CURRENT_DATE + 15,'High',
           'Tủ chống cháy, thông gió');

INSERT INTO certification
VALUES (1,'An toàn phòng thí nghiệm cơ bản','Trung tâm An toàn PTN',12),
       (2,'Xử lý dung môi dễ cháy','Trung tâm An toàn PTN',12),
       (3,'Xử lý axit oxy hóa mạnh','Trung tâm An toàn PTN',6);

INSERT INTO has_certification
VALUES (1,1,CURRENT_DATE-100,CURRENT_DATE + 265,'Valid'),
       (2,1,CURRENT_DATE-100,CURRENT_DATE + 265,'Valid'),
       (2,2,CURRENT_DATE-100,CURRENT_DATE + 265,'Valid'),
       (2,3,CURRENT_DATE-60,CURRENT_DATE + 120,'Valid'),
       (4,1,CURRENT_DATE-100,CURRENT_DATE + 265,'Valid'),
       (5,1,CURRENT_DATE-100,CURRENT_DATE + 265,'Valid'),
       (5,2,CURRENT_DATE-100,CURRENT_DATE + 265,'Valid'),
       (6,1,CURRENT_DATE-400,CURRENT_DATE-35,'Valid'),
       (9,1,CURRENT_DATE-100,CURRENT_DATE + 265,'Valid'),
       (9,2,CURRENT_DATE-100,CURRENT_DATE + 265,'Valid'),
```

```
       (10,1,CURRENT_DATE-100,CURRENT_DATE + 265,'Valid'),
       (10,2,CURRENT_DATE-100,CURRENT_DATE + 265,'Valid'),
       (10,3,CURRENT_DATE-60,CURRENT_DATE + 120,'Valid'),
       (11,1,CURRENT_DATE-100,CURRENT_DATE + 265,'Valid'),
       (12,1,CURRENT_DATE-100,CURRENT_DATE + 265,'Valid'),
       (12,2,CURRENT_DATE-400,CURRENT_DATE-35,'Valid');

INSERT INTO chemical_certification
VALUES(1,2),
      (2,1),
      (2,3),
      (5,2),
      (6,2);

INSERT INTO allocated_equip
VALUES(1,7),
      (1,8),
      (2,4),
      (2,5),
      (2,6),
      (3,1),
      (3,2),
      (3,3),
      (3,11),
      (4,1);

INSERT INTO allocated_chem
VALUES(1,1),
      (1,2),
      (1,3),
      (1,4),
      (1,5),
      (1,6),
      (2,3);

SELECT receive_chemical(1,
           10, 20, 'Nhập kho đầu kỳ');

SELECT receive_chemical(2,
           10, 5, 'Nhập kho đầu kỳ');

SELECT receive_chemical(3,
           10, 10, 'Nhập kho đầu kỳ');

SELECT receive_chemical(4,
           10, 1000, 'Nhập kho đầu kỳ');

SELECT receive_chemical(5,
           10, 3, 'Tiếp nhận tồn cần cách ly');

SELECT receive_chemical(6,
           10, 8, 'Nhập kho đầu kỳ');

SELECT issue_chemical(1,
           5, 1, 2.5, 'Tổng hợp mẫu cảm biến');

SELECT issue_chemical(2,
           10, 1, 0.5, 'Chuẩn bị phản ứng có kiểm soát');
```

```
SELECT issue_chemical(3,
          4, 2, 1, 'Pha dung dịch chuẩn');

SELECT issue_chemical(4,
          2, 1, 50, 'Chuẩn bị mẫu phân tích');

INSERT INTO access_log(person_id,
                lab_id, entry_time, exit_time, purpose, approved_by)
VALUES (4,1,CURRENT_DATE-3 + time '08:00',CURRENT_DATE-3 + time '11:00',
    'Thu thập dữ liệu',NULL),
       (4,2,CURRENT_DATE-2 + time '18:00',CURRENT_DATE-2 + time '20:00',
           'Đo kiểm theo lịch bổ sung',2),
       (5,3,CURRENT_DATE-2 + time '09:00',CURRENT_DATE-2 + time '11:30',
           'Tổng hợp mẫu',NULL),
       (10,3,CURRENT_DATE-2 + time '22:30',CURRENT_DATE-1 + time '00:30',
           'Kiểm tra an toàn cuối ca',NULL),
       (2,3,CURRENT_DATE-1 + time '09:00',CURRENT_DATE-1 + time '12:00',
           'Giám sát phản ứng',NULL),
       (1,1,CURRENT_DATE-1 + time '23:00',CURRENT_DATE + time '00:10',
           'Khắc phục sự cố máy chủ',NULL),
       (9,2,CURRENT_DATE + time '06:00',NULL,'Kiểm tra thiết bị trước ca',NULL);

INSERT INTO borrow_transaction(borrow_id,
                person_id, equipment_id, borrow_date, expected_return_date,
                actual_return_date)
VALUES (1,4,1,CURRENT_DATE-10,CURRENT_DATE-5,NULL),
       (2,4,2,CURRENT_DATE-2,CURRENT_DATE + 5,NULL),
       (3,5,4,CURRENT_DATE-20,CURRENT_DATE-15,CURRENT_DATE-14),
       (4,6,3,CURRENT_DATE-8,CURRENT_DATE-3,CURRENT_DATE-4),
       (5,5,5,CURRENT_DATE-1,CURRENT_DATE + 6,NULL);

INSERT INTO maintenance_log(maintenance_id,
                technician_id, equipment_id, start_date, end_date,
                description, cost)
VALUES (1,9,6,CURRENT_DATE-2,NULL,'Hiệu chuẩn bước sóng',1500000),
       (2,11,11,CURRENT_DATE-30,CURRENT_DATE-29,'Thay cáp và kiểm tra kết nối',
           300000),
       (3,9,4,CURRENT_DATE-60,CURRENT_DATE-59,'Vệ sinh hệ quang học',700000);

SELECT setval(pg_get_serial_sequence('person',
                  'person_id'),
                  (SELECT max(person_id)
                   FROM person));

SELECT setval(pg_get_serial_sequence('laboratory',
                  'lab_id'),
                  (SELECT max(lab_id)
                   FROM laboratory));

SELECT setval(pg_get_serial_sequence('research_grant',
                  'grant_id'),
                  (SELECT max(grant_id)
                   FROM research_grant));

SELECT setval(pg_get_serial_sequence('research_point',
                  'rearch_point_id'),
                  (SELECT max(rearch_point_id)
```

```
                    FROM research_point));

SELECT setval(pg_get_serial_sequence('equipment',
                   'equipment_id'),
                   (SELECT max(equipment_id)
                    FROM equipment));

SELECT setval(pg_get_serial_sequence('chemical',
                   'chemical_id'),
                   (SELECT max(chemical_id)
                    FROM chemical));

SELECT setval(pg_get_serial_sequence('certification',
                   'cert_id'),
                   (SELECT max(cert_id)
                    FROM certification));

SELECT setval(pg_get_serial_sequence('borrow_transaction',
                   'borrow_id'),
                   (SELECT max(borrow_id)
                    FROM borrow_transaction));

SELECT setval(pg_get_serial_sequence('maintenance_log',
                   'maintenance_id'),
                   (SELECT max(maintenance_id)
                    FROM maintenance_log));
```

4.2 Truy vấn nâng cao và hiệu năng

Tập 12 truy vấn sử dụng JOIN nhiều bảng, truy vấn con tương quan, NOT EXISTS, tổng hợp có điều kiện, CTE và window function. Các kết quả lấy trực tiếp từ lần chạy thực tế trên dữ liệu mẫu; giá trị hiển thị không phải dữ liệu dự đoán.

Chỉ mục hỗ trợ các đường truy cập thường gặp: research_point(grant_id), equipment(lab_id), chemical(lab_id,expiry_date), has_certification(person_id,expiry_date), access_log(lab_id,entry_time DESC), lịch sử theo người/thiết bị và movement theo hóa chất/thời điểm. Partial index phục vụ lượt chưa trả và bảo trì chưa kết thúc.

Mục 4.3 ghi kết quả EXPLAIN (ANALYZE, BUFFERS) và thời gian thực đo trên bộ seed. Dữ liệu nhỏ nên chưa kết luận chỉ mục tăng tốc hoặc đạt SLA. Các view tổng hợp tách nhánh M:N trước khi tính tổng để tránh đếm lặp; cần đo lại trên dữ liệu đại diện trước khi điều chỉnh chỉ mục cho môi trường triển khai.

4.2.1 Q01 Tổng hợp đề tài đang thực hiện

Kết hợp research_grant, person và v_grant_asset_summary để xem chủ nhiệm, ngân sách còn lại, tỷ lệ đã sử dụng, số thiết bị và hóa chất duy nhất. Hai truy vấn con độc lập trong view tránh nhân bản tổng khi một hạng mục có nhiều thiết bị và nhiều hóa chất.

Kỹ thuật sử dụng: INNER JOIN, truy vấn con tương quan, DISTINCT, phép tính tỷ lệ, NULLIF.

```
SET search_path = lab_tracker, pg_catalog;

SELECT g.grant_id,g.grant_title,p.full_name AS principal_investigator,
       g.remaining_budget,
       round(100 * (g.total_budget - g.remaining_budget) / nullif(g.total_budget,
            0), 2) AS used_budget_pct, s.equipment_count,s.chemical_count
FROM research_grant g
JOIN person p ON p.person_id = g.principal_investigator_id
JOIN v_grant_asset_summary s USING(grant_id)
WHERE g.status = 'Active'
ORDER BY g.grant_id;
```

Kết quả thực thi: 2 dòng. Hai đề tài Active có tỷ lệ sử dụng ngân sách 36% và 24%; số thiết bị tương ứng 5 và 4, số hóa chất 6 và 0.

Dòng	Dữ liệu trả về
1	grant_id = 1 grant_title = Phân tích vật liệu cảm biến principal_investigator = Trần Thu Bình remaining_budget = 320000000.00 used_budget_pct = 36.00 equipment_count = 5 chemical_count = 6
2	grant_id = 2 grant_title = Hệ thống giám sát phòng thí nghiệm principal_investigator = Nguyễn Minh An remaining_budget = 190000000.00 used_budget_pct = 24.00 equipment_count = 4 chemical_count = 0

4.2.2 Q02 Người đang giữ thiết bị và số lượt quá hạn

Đếm tất cả lượt chưa trả bằng actual_return_date IS NULL, bao gồm cả lượt có effective status Overdue; không chỉ đếm trạng thái Borrowed.

Kỹ thuật sử dụng: LEFT JOIN, GROUP BY, HAVING, COUNT FILTER.

```
SET search_path = lab_tracker, pg_catalog;

SELECT r.person_id,r.full_name,r.person_role,count(b.borrow_id) AS open_loans,
       count(b.borrow_id) FILTER (
            WHERE b.expected_return_date < CURRENT_DATE) AS overdue_loans
FROM v_person_role r
LEFT JOIN borrow_transaction b ON b.person_id = r.person_id
AND b.actual_return_date IS NULL
GROUP BY r.person_id,r.full_name,r.person_role
HAVING count(b.borrow_id) > 0
ORDER BY r.person_id;
```

Kết quả thực thi: 2 dòng. Người 4 đang giữ 2 thiết bị, gồm 1 quá hạn; người 5 đang giữ 1 thiết bị và chưa quá hạn.

Dòng	Dữ liệu trả về
1	person_id = 4 full_name = Phạm Hải Dương person_role = Student open_loans = 2 overdue_loans = 1
2	person_id = 5 full_name = Võ Ngọc Hân person_role = Student open_loans = 1 overdue_loans = 0

4.2.3 Q03 Hóa chất sắp hết hạn trong 30 ngày

Liên kết hóa chất với phòng lưu trữ, lấy tồn còn dương và hạn còn trong 30 ngày; hóa chất đã quá hạn được tách khỏi danh sách sắp hết hạn.

Kỹ thuật sử dụng: JOIN, BETWEEN, điều kiện kết hợp, số học ngày.

```
SET search_path = lab_tracker, pg_catalog;

SELECT c.chemical_id,c.chemical_name,l.lab_name,c.quantity_in_stock,c.unit,
       c.expiry_date - CURRENT_DATE AS days_to_expiry
FROM chemical c
JOIN laboratory l USING(lab_id)
WHERE c.expiry_date BETWEEN CURRENT_DATE AND CURRENT_DATE + 30
    AND c.quantity_in_stock > 0
ORDER BY c.expiry_date,c.chemical_id;
```

Kết quả thực thi: 2 dòng. Acetone còn 8 L và 15 ngày; Đồng sulfat còn 950 g và 25 ngày.

Dòng	Dữ liệu trả về
1	chemical_id = 6 chemical_name = Acetone lab_name = Phòng phản ứng hóa học quantity_in_stock = 8.000 unit = L days_to_expiry = 15

Dòng	Dữ liệu trả về
2	chemical_id = 4 chemical_name = Đồng sulfat lab_name = Phòng phân tích vật liệu quantity_in_stock = 950.000 unit = g days_to_expiry = 25

4.2.4 Q04 Người Active chưa có chứng nhận an toàn hợp lệ

Phép phủ định EXISTS tìm người không có bất kỳ chứng nhận nào có trạng thái Valid và thời điểm hiện tại nằm trong khoảng cấp–hết hạn.

Kỹ thuật sử dụng: NOT EXISTS tương quan, kiểm tra thời hạn.

```
SET search_path = lab_tracker, pg_catalog;

SELECT p.person_id,p.full_name
FROM person p
WHERE p.status = 'Active'
    AND NOT EXISTS
        (SELECT 1
         FROM has_certification h
         WHERE h.person_id = p.person_id
             AND h.status = 'Valid'
             AND CURRENT_DATE BETWEEN h.issue_date AND h.expiry_date)
ORDER BY p.person_id;
```

Kết quả thực thi: 3 dòng. Các mã 3, 6, 7; người 6 có chứng nhận nhưng đã hết hạn.

Dòng	Dữ liệu trả về
1	person_id = 3 full_name = Lê Quang Chi
2	person_id = 6 full_name = Đặng Đức Khoa
3	person_id = 7 full_name = Bùi Thanh Lam

4.2.5 Q05 Lượt truy cập và tổng giờ người theo phòng

Tính số lượt trong bảy ngày và tổng thời gian của các lượt đã có exit_time. Lượt chưa đóng vẫn được đếm nhưng chưa đóng góp thời lượng.

Kỹ thuật sử dụng: LEFT JOIN, tổng hợp thời gian, EXTRACT, COALESCE.

```sql
SET search_path = lab_tracker, pg_catalog;

SELECT l.lab_id,l.lab_name,count(a.log_id) AS visits,
       round(coalesce(sum(extract(epoch
            FROM(a.exit_time - a.entry_time)) / 3600), 0)::numeric, 2) AS
                completed_person_hours
FROM laboratory l
LEFT JOIN access_log a ON a.lab_id = l.lab_id
AND a.entry_time >= CURRENT_DATE-7
GROUP BY l.lab_id,l.lab_name
ORDER BY l.lab_id;
```

Kết quả thực thi: 3 dòng. Phòng 1: 2 lượt, 4,17 giờ; phòng 2: 2 lượt, 2 giờ; phòng 3: 3 lượt, 7,5 giờ.

Dòng	Dữ liệu trả về
1	lab_id = 1 lab_name = Phòng máy tính nghiên cứu visits = 2 completed_person_hours = 4.17
2	lab_id = 2 lab_name = Phòng phân tích vật liệu visits = 2 completed_person_hours = 2.00
3	lab_id = 3 lab_name = Phòng phản ứng hóa học visits = 3 completed_person_hours = 7.50

4.2.6 Q06 Tình trạng thiết bị theo phòng

Dùng tổng hợp có điều kiện để so sánh bốn trạng thái thiết bị trong từng phòng, vẫn hiển thị phòng không có thiết bị.

Kỹ thuật sử dụng: LEFT JOIN, COUNT FILTER, GROUP BY.

```sql
SET search_path = lab_tracker, pg_catalog;

SELECT l.lab_id,l.lab_name,count(e.equipment_id) AS total_equipment,
       count(e.equipment_id) FILTER(
            WHERE e.current_status = 'Available') AS available,
       count(e.equipment_id) FILTER(
            WHERE e.current_status = 'In Use') AS in_use,
       count(e.equipment_id) FILTER(
            WHERE e.current_status = 'Under Maintenance') AS under_maintenance,
       count(e.equipment_id) FILTER(
            WHERE e.current_status = 'Retired') AS retired
FROM laboratory l
LEFT JOIN equipment e USING(lab_id)
GROUP BY l.lab_id,l.lab_name
ORDER BY l.lab_id;
```

Kết quả thực thi: 3 dòng. Phòng 1 có 6 thiết bị; phòng 2 có 4; phòng 3 có 2. Một thiết bị Under Maintenance và một Retired.

Dòng	Dữ liệu trả về
1	lab_id = 1 lab_name = Phòng máy tính nghiên cứu total_equipment = 6 available = 3 in_use = 2 under_maintenance = 0 retired = 1
2	lab_id = 2 lab_name = Phòng phân tích vật liệu total_equipment = 4 available = 2 in_use = 1 under_maintenance = 1 retired = 0
3	lab_id = 3 lab_name = Phòng phản ứng hóa học total_equipment = 2 available = 2 in_use = 0 under_maintenance = 0 retired = 0

4.2.7 Q07 Xếp hạng chi phí bảo trì trong từng phòng

CTE tổng hợp chi phí lịch sử cho từng thiết bị, sau đó DENSE_RANK phân hạng trong mỗi phòng. Chỉ hiển thị thiết bị có chi phí.

Kỹ thuật sử dụng: CTE, LEFT JOIN, SUM, DENSE_RANK OVER PARTITION BY.

```
SET search_path = lab_tracker, pg_catalog;

WITH costs AS
    (SELECT e.equipment_id,e.equipment_name,e.lab_id,
            coalesce(sum(m.cost), 0) AS maintenance_cost
     FROM equipment e
     LEFT JOIN maintenance_log m USING(equipment_id)
     GROUP BY e.equipment_id),
     ranked AS
    (SELECT *,
            dense_rank() OVER(PARTITION BY lab_id
            ORDER BY maintenance_cost DESC) AS cost_rank
     FROM costs)
SELECT equipment_id,equipment_name,lab_id,maintenance_cost,cost_rank
FROM ranked
WHERE maintenance_cost > 0
ORDER BY lab_id,cost_rank,equipment_id;
```

Kết quả thực thi: 3 dòng. Thiết bị 11: 300.000; thiết bị 6: 1.500.000; thiết bị 4: 700.000 VND.

Dòng	Dữ liệu trả về
1	equipment_id = 11 equipment_name = Bộ thu thập dữ liệu lab_id = 1 maintenance_cost = 300000.00 cost_rank = 1
2	equipment_id = 6 equipment_name = Máy đo phổ UV-Vis lab_id = 2 maintenance_cost = 1500000.00 cost_rank = 1
3	equipment_id = 4 equipment_name = Kính hiển vi quang học lab_id = 2 maintenance_cost = 700000.00 cost_rank = 2

4.2.8 Q08 Danh sách có đủ chứng nhận sử dụng hóa chất High/Extreme

Tạo các cặp người–hóa chất; yêu cầu có chính sách chứng nhận và không tồn tại chứng nhận bắt buộc mà người đó thiếu hoặc đã hết hạn. Đây là phép chia quan hệ bằng hai lớp NOT EXISTS.

Kỹ thuật sử dụng: CROSS JOIN, EXISTS, NOT EXISTS lồng nhau, kiểm tra hiệu lực.

```
SET search_path = lab_tracker, pg_catalog;

SELECT p.person_id,p.full_name,c.chemical_id,c.chemical_name
FROM person p
CROSS JOIN chemical c
WHERE p.status = 'Active'
    AND c.hazard_level IN('High','Extreme')
    AND c.expiry_date >= CURRENT_DATE
    AND EXISTS
        (SELECT 1
         FROM chemical_certification r
         WHERE r.chemical_id = c.chemical_id)
    AND NOT EXISTS
        (SELECT 1
         FROM chemical_certification r
         WHERE r.chemical_id = c.chemical_id
             AND NOT EXISTS
                 (SELECT 1
                  FROM has_certification h
                  WHERE h.person_id = p.person_id
            AND h.cert_id = r.cert_id
            AND h.status = 'Valid'
            AND CURRENT_DATE BETWEEN h.issue_date AND h.expiry_date))
ORDER BY p.person_id,c.chemical_id;
```

Kết quả thực thi: 10 dòng. 10 cặp đủ điều kiện; chỉ người 2 và 10 có đủ chứng nhận cho Axit nitric Extreme.

Dòng	Dữ liệu trả về
1	person_id = 2 full_name = Trần Thu Bình chemical_id = 1 chemical_name = Ethanol 96%
2	person_id = 2 full_name = Trần Thu Bình chemical_id = 2 chemical_name = Axit nitric 65%
3	person_id = 2 full_name = Trần Thu Bình chemical_id = 6 chemical_name = Acetone
4	person_id = 5 full_name = Võ Ngọc Hân chemical_id = 1 chemical_name = Ethanol 96%
5	person_id = 5 full_name = Võ Ngọc Hân chemical_id = 6 chemical_name = Acetone

Dòng	Dữ liệu trả về
6	person_id = 9 full_name = Nguyễn Hoàng Nam chemical_id = 1 chemical_name = Ethanol 96%
7	person_id = 9 full_name = Nguyễn Hoàng Nam chemical_id = 6 chemical_name = Acetone
8	person_id = 10 full_name = Trần Bảo Ngọc chemical_id = 1 chemical_name = Ethanol 96%
9	person_id = 10 full_name = Trần Bảo Ngọc chemical_id = 2 chemical_name = Axit nitric 65%
10	person_id = 10 full_name = Trần Bảo Ngọc chemical_id = 6 chemical_name = Acetone

4.2.9 Q09 Đối soát tồn kho với sổ nhập xuất

Tổng hợp Receipt và Issue theo từng hóa chất, so sánh số tồn lưu với số dư tính từ ledger. Không cộng chung các đơn vị khác nhau.

Kỹ thuật sử dụng: JOIN, SUM FILTER, GROUP BY, view tổng hợp, đối soát.

```
SET search_path = lab_tracker, pg_catalog;

SELECT c.chemical_id,c.chemical_name,c.unit,
       sum(m.quantity) FILTER(
            WHERE m.movement_type = 'Receipt') AS total_received,
       coalesce(sum(m.quantity) FILTER(
            WHERE m.movement_type = 'Issue'), 0) AS total_issued,
                c.quantity_in_stock,a.ledger_balance,a.reconciled
FROM chemical c
JOIN chemical_movement m USING(chemical_id)
JOIN v_chemical_stock_audit a USING(chemical_id)
GROUP BY c.chemical_id,a.ledger_balance,a.reconciled
ORDER BY c.chemical_id;
```

Kết quả thực thi: 6 dòng. Sáu hóa chất đều reconciled=true; Ethanol 17,500 L, Axit nitric 4,500 L, Natri clorua 9 kg, Đồng sulfat 950 g.

Dòng	Dữ liệu trả về
1	chemical_id = 1 chemical_name = Ethanol 96% unit = L total_received = 20.000 total_issued = 2.500 quantity_in_stock = 17.500 ledger_balance = 17.500 reconciled = True
2	chemical_id = 2 chemical_name = Axit nitric 65% unit = L total_received = 5.000 total_issued = 0.500 quantity_in_stock = 4.500 ledger_balance = 4.500 reconciled = True
3	chemical_id = 3 chemical_name = Natri clorua unit = kg total_received = 10.000 total_issued = 1.000 quantity_in_stock = 9.000 ledger_balance = 9.000 reconciled = True
4	chemical_id = 4 chemical_name = Đồng sulfat unit = g total_received = 1000.000 total_issued = 50.000 quantity_in_stock = 950.000 ledger_balance = 950.000 reconciled = True
5	chemical_id = 5 chemical_name = Methanol unit = L total_received = 3.000 total_issued = 0 quantity_in_stock = 3.000 ledger_balance = 3.000 reconciled = True
6	chemical_id = 6 chemical_name = Acetone unit = L total_received = 8.000 total_issued = 0 quantity_in_stock = 8.000 ledger_balance = 8.000 reconciled = True

4.2.10 Q10 Đề tài có đủ thiết bị cho mọi hạng mục đang Active

Phủ định hai lớp biểu diễn: không có hạng mục Active nào thiếu thiết bị. Đề tài không có hạng mục Active thỏa điều kiện theo lượng từ phổ dụng; giao diện có thể thêm điều kiện EXISTS nếu yêu cầu bắt buộc tồn tại.

Kỹ thuật sử dụng: Phép chia quan hệ, NOT EXISTS lồng nhau.

```
SET search_path = lab_tracker, pg_catalog;

SELECT g.grant_id,g.grant_title
FROM research_grant g
WHERE g.status = 'Active'
    AND NOT EXISTS
        (SELECT 1
         FROM research_point p
         WHERE p.grant_id = g.grant_id
             AND p.status = 'Active'
             AND NOT EXISTS
                 (SELECT 1
                  FROM allocated_equip a
                  WHERE a.rearch_point_id = p.rearch_point_id))
ORDER BY g.grant_id;
```

Kết quả thực thi: 2 dòng. Hai đề tài Active mã 1 và 2 đều có thiết bị cho mọi hạng mục đang thực hiện.

Dòng	Dữ liệu trả về
1	grant_id = 1 grant_title = Phân tích vật liệu cảm biến
2	grant_id = 2 grant_title = Hệ thống giám sát phòng thí nghiệm

4.2.11 Q11 Kiểm toán sinh viên truy cập ngoài giờ

Kết hợp access_log, student, laboratory và hai lần person để xem người truy cập và người phê duyệt; định dạng thời điểm Việt Nam thành chuỗi để giữ nguyên giờ địa phương.

Kỹ thuật sử dụng: Nhiều JOIN, alias hai vai trò người, lọc thời gian.

```
SET search_path = lab_tracker, pg_catalog;

SELECT a.log_id,p.full_name,l.lab_name,
       to_char(a.entry_time, 'YYYY-MM-DD HH24:MI:SS') AS entry_time,a.approved_by,
           q.full_name AS approver
FROM access_log a
JOIN student s ON s.student_id = a.person_id
JOIN person p USING(person_id)
JOIN laboratory l USING(lab_id)
JOIN person q ON q.person_id = a.approved_by
WHERE a.entry_time::time < time '07:00'
    OR a.entry_time::time >= time '17:00'
ORDER BY a.log_id;
```

Kết quả thực thi: 1 dòng. Một lượt mã 2: Phạm Hải Dương vào phòng phân tích lúc 18:00, được Trần Thu Bình phê duyệt.

Dòng	Dữ liệu trả về
1	log_id = 2 full_name = Phạm Hải Dương lab_name = Phòng phân tích vật liệu entry_time = 2026-10-19 18:00:00 approved_by = 2 approver = Trần Thu Bình

4.2.12 Q12 Mức sử dụng và tỷ lệ quá hạn của thiết bị

CTE gom lượt mượn theo thiết bị, đếm quá hạn, tính tỷ lệ và xếp hạng theo số lượt. Late count dùng trạng thái đã chuẩn hóa khi tạo/trả giao dịch.

Kỹ thuật sử dụng: CTE, COUNT FILTER, NULLIF, RANK OVER.

```
SET search_path = lab_tracker, pg_catalog;

WITH usage AS
    (SELECT e.equipment_id,e.equipment_name,count(b.borrow_id) AS loan_count,
            count(b.borrow_id) FILTER(
            WHERE b.status = 'Overdue') AS late_count
     FROM equipment e
     LEFT JOIN borrow_transaction b USING(equipment_id)
     GROUP BY e.equipment_id)
SELECT equipment_id,equipment_name,loan_count,late_count,
       round(100.0 * late_count / nullif(loan_count,
            0), 2) AS late_pct, rank() OVER(
            ORDER BY loan_count DESC) AS usage_rank
FROM usage
WHERE loan_count > 0
ORDER BY usage_rank,equipment_id;
```

Kết quả thực thi: 5 dòng. Năm thiết bị từng có giao dịch; thiết bị 1 và 4 có tỷ lệ quá hạn 100% trong bộ dữ liệu mẫu nhỏ.

Dòng	Dữ liệu trả về
1	equipment_id = 1 equipment_name = Máy trạm nghiên cứu 01 loan_count = 1 late_count = 1 late_pct = 100.00 usage_rank = 1
2	equipment_id = 2 equipment_name = Máy trạm nghiên cứu 02 loan_count = 1 late_count = 0 late_pct = 0.00 usage_rank = 1
3	equipment_id = 3 equipment_name = Bộ cảm biến môi trường loan_count = 1 late_count = 0 late_pct = 0.00 usage_rank = 1
4	equipment_id = 4 equipment_name = Kính hiển vi quang học loan_count = 1 late_count = 1 late_pct = 100.00 usage_rank = 1
5	equipment_id = 5 equipment_name = Cân phân tích 01 loan_count = 1 late_count = 0 late_pct = 0.00 usage_rank = 1

4.3 Đánh giá hiệu năng

4.3.1 Kết quả thực đo trên bộ seed

Ngày 22/10/2026, run_query_performance.mjs tạo một PGlite 0.5.10 trong bộ nhớ (PostgreSQL 18.6), nạp nguyên bản 01_schema.sql, 02_seed.sql và 04_rbac.sql, chạy ANALYZE rồi thực hiện 12 truy vấn của 03_queries.sql với role lab_manager. Máy đo: Windows x64, Intel Core i5-13420H; Node v24.19.0; múi giờ Asia/Ho_Chi_Minh. Bộ seed có 12 người, 12 thiết bị, 6 hóa chất và 7 lượt truy cập. Không dùng cơ sở dữ liệu bên ngoài và không lưu thay đổi vào nguồn.

Mỗi truy vấn chạy khởi động 3 lần, sau đó đo xen kẽ 15 lần trực tiếp và 15 lần EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON, TIMING ON, SUMMARY ON). Tổng cộng 180 lần đo trực tiếp và 180 kế hoạch thực thi; kết quả của cả 12

truy vấn ổn định qua các lần chạy. Median client và p95 client đo bằng performance.now(), gồm cầu nối JavaScript/WASM, lập kế hoạch, thực thi và tạo kết quả. Median EXPLAIN là Execution Time do PostgreSQL báo, có chi phí gắn thiết bị đo và không gồm cầu nối JavaScript/WASM; hai cột không đồng nhất.

Truy vấn	Số dòng	Median client (ms)	p95 client (ms)	Median EXPLAIN execution (ms)
Q01	2	0.9024	1.5510	0.159
Q02	2	0.9458	1.5575	0.125
Q03	2	0.5050	0.6707	0.046
Q04	3	0.4644	0.9575	0.051
Q05	3	0.5474	0.9871	0.072
Q06	3	0.5484	1.0689	0.072
Q07	3	0.6990	0.9983	0.111
Q08	10	0.8756	1.1083	0.133
Q09	6	0.9031	2.3423	0.146
Q10	2	0.4459	0.5736	0.049
Q11	1	1.0092	1.3343	0.092
Q12	5	0.6263	0.7513	0.094

Trong lần đo này, median client cao nhất là 1,0092 ms (Q11), p95 cao nhất 2,3423 ms (Q09) và median EXPLAIN execution cao nhất 0,159 ms (Q01). p95 dùng thứ hạng gần nhất của 15 mẫu, chỉ mô tả phép thử nhỏ này. Không có ngưỡng thời gian dùng để đánh dấu đạt/trượt; kết quả đạt nghĩa là truy vấn thực thi thành công, kết quả ổn định và nguồn SQL không đổi.

Các trường hợp nâng cao có kế hoạch thực tế: Q01 dùng Hash Join và Aggregate qua view đếm tài sản; Q07/Q12 có WindowAgg cho xếp hạng; Q08/Q10 kiểm tra phủ định lồng nhau qua các phép join; Q09 tổng hợp ledger. Kế hoạch đều dùng Seq Scan trên các bảng rất nhỏ, không có shared read hoặc temp write block trong các mẫu đo. Điều này chưa chứng minh chỉ mục không hữu ích khi quy mô tăng.

Chưa đo so sánh bật/tắt chỉ mục hoặc độ chọn lọc trên dữ liệu đại diện. Index chemical(lab_id,expiry_date) hỗ trợ bộ lọc có lab_id đứng đầu, còn Q03 lọc hạn dùng xuyên các phòng; chỉ cân nhắc index bắt đầu bằng expiry_date sau khi có phép đo quy mô phù hợp. Thời gian PGlite một phiên, dữ liệu nóng và seed nhỏ không xác nhận SLA 200 ms, hiệu năng PostgreSQL máy chủ, mạng hoặc tải nhiều người dùng. T14/H13 nhiều kết nối và website/API/xác thực của Phase 4 chưa được chạy bởi phép đo này.

Tái lập từ thư mục gốc bằng node work/completeness_review/run_query_performance.mjs khi dependency PGlite cục bộ đã sẵn sàng. query_performance_results.json lưu mẫu thời gian thô, kế hoạch JSON đầy đủ của lượt đầu, actual rows/loops, thống kê buffer, chỉ mục và SHA-256 trước/sau của bốn file SQL; mọi hash đều giữ nguyên. query_performance_notes.md ghi cách chạy và giới hạn.

4.3.2 Các phép đo quy mô lớn và đồng thời cần thực hiện

Các tình huống dưới đây là kế hoạch tiếp theo, chưa có số đo được xác nhận. Chúng dùng dữ liệu đại diện và PostgreSQL máy chủ để đánh giá tính mở rộng, chỉ mục, I/O lạnh và cạnh tranh ghi; không suy rộng số đo seed thành kết quả của các tình huống này.

Kịch bản	Cách đo	Tiêu chí đánh giá
Tra cứu thiết bị	Nạp 100000 thiết bị tổng hợp, 30% Available; lọc theo lab_id và current_status, LIMIT 20. Chạy EXPLAIN (ANALYZE, BUFFERS), ghi kế hoạch và số hàng thực đọc.	p95 dưới 200 ms trên môi trường được ghi nhận; so sánh trước và sau chỉ mục phù hợp.
Lịch sử truy cập	Nạp 1000000 access_log; lọc một phòng trong 30 ngày, ORDER BY entry_time DESC, LIMIT 50.	Số buffer và hàng đọc phù hợp độ chọn lọc; kiểm tra ix_access_lab_entry được dùng khi có lợi.
Cạnh tranh ghi	Hai kết nối thật chạy T14 và H13 bằng 06_concurrency.mjs.	Chỉ một giao dịch được chấp nhận trong mỗi cặp; không vượt giới hạn ba thiết bị hoặc làm âm tồn.
Báo cáo quản trị	Chạy Q01 và Q05 với nhiều hạng mục và nhiều phân bổ trên mỗi đề tài.	Tổng không bị nhân do JOIN nhiều nhánh M:N; lưu cả kết quả đúng và kế hoạch truy vấn.

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT equipment_id, equipment_name, current_status
FROM lab_tracker.equipment
WHERE lab_id = 2
    AND current_status = 'Available'
ORDER BY equipment_id
LIMIT 20;
```

5 Kiểm chứng và bảo mật

5.1 Kết quả xác minh

Lần chạy thực tế xác nhận tạo được 19 bảng, 5 view, 23 trigger; thực thi đủ 12 truy vấn; 44/44 kiểm thử nghiệp vụ và 10/10 kiểm thử phân quyền đạt. Các ca thử bao phủ bộ T01–T13 của Phase 2, tăng cường biên thời gian, sửa mã lớp con, đúng/sai loại chứng nhận, ledger và quyền ghi.

Bốn tình huống được rà soát bổ sung trước khi chốt gồm: thay đổi mã lớp con làm mất chuyên môn hóa toàn phần; truy cập đúng 17:00; chứng nhận thay đổi đồng thời với vào phòng; tài sản chuyển phòng đồng thời với xóa phòng của đề tài. Các sửa chữa đã được thực thi lại bằng toàn bộ tập kiểm thử tuần tự.

T14 và H13 là kiểm thử đồng thời độc lập, đã có script nhưng chưa có kết quả chạy PostgreSQL nhiều kết nối. Kiểm chứng API, xác thực người dùng, phê duyệt thật, lịch làm mới trạng thái, trình diễn nhóm và đánh giá đồng đẳng thuộc Phase 4; không được ghi nhận hoàn thành chỉ từ script SQL hoặc kế hoạch tích hợp.

Nội dung	Kết quả đã chạy
Môi trường	PostgreSQL 18.6 (PGlite 0.5.10) on wasm32-unknown-emscripten, compiled by emcc (Emscripten gcc/clang-like replacement + linker emulating GNU ld) 6.0.5 (1db513782be24469589d7cb8a1f1834e9a33f271), 32-bit
Múi giờ	Asia/Ho_Chi_Minh
Bảng và view	19 bảng và 5 view
Trigger	23
Truy vấn	12 truy vấn thực thi thành công
Kiểm thử nghiệp vụ	44 trên 44 đạt
Kiểm thử phân quyền	10 trên 10 đạt
Đo truy vấn trên seed	12/12 truy vấn ổn định; 180 lần chạy trực tiếp và 180 EXPLAIN ANALYZE. Xem mục 4.3.1.
Hai kết nối đồng thời	Đã chuẩn bị 06_concurrency.mjs; chưa thực chạy trên PostgreSQL máy chủ.

5.1.1 Truy vết quy tắc nghiệp vụ

Mã	Quy tắc	Cơ chế và bằng chứng
BR01	Mỗi PERSON thuộc đúng một lớp con	FK lớp con + bốn constraint trigger DEFERRABLE INITIALLY DEFERRED; chỉ commit khi tổng số lớp con bằng 1. Bốn trigger bổ sung giữ bất biến mã người/lớp con. Kiểm thử: T01A–T01D
BR02	PI phải là Faculty	principal_investigator_id NOT NULL tham chiếu faculty.faculty_id. Kiểm thử: T02A
BR03	Chỉ Technician tạo/cập nhật bảo trì	FK technician_id; không cấp DML trực tiếp maintenance_log; chỉ role lab_technician được thực thi start_maintenance và finish_maintenance, các hàm còn kiểm tra actor Active thuộc Technician. Kiểm thử: T02B,H12,S08,S09
BR04	Sinh viên ngoài giờ cần phê duyệt	enforce_access kiểm tra khung [07:00,17:00); approved_by phải là người Active khác, thuộc Faculty hoặc Technician. Backend phải gắn mã người phê duyệt từ yêu cầu đã được duyệt thật. Kiểm thử: T03A–T03D
BR05	Restricted/Hazardous yêu cầu chứng nhận	Trigger truy cập kiểm tra status Valid và issue_date ≤ ngày vào ≤ expiry_date; khóa chia sẻ chứng nhận trong giao dịch. Kiểm thử: T04,T05C
BR06	High/Extreme yêu cầu đúng chứng nhận	chemical_certification biểu diễn tập chứng nhận bắt buộc. issue_chemical chỉ nhận nếu chính sách không rỗng và người dùng có đủ mọi chứng nhận còn hiệu lực. Kiểm thử: H01–H03,H09
BR07	Chứng nhận hết hạn chuyển Expired	sync_cert_status chuẩn hóa khi ghi. refresh_temporal_status cập nhật theo thời gian; v_certification_current và nghiệp vụ luôn kiểm tra ngày nên chứng nhận hết hạn không được dùng giữa các lượt chạy lịch. Tầng tích hợp phải gọi hàm hằng ngày. Kiểm thử: T05B,T05C
BR08	Ngày hết hạn chứng nhận sau ngày cấp	CHECK(expiry_date>issue_date) và NOT NULL. Kiểm thử: T05A
BR09	Đề tài có chủ nhiệm	FK NOT NULL tới faculty triển khai đúng một PI theo Phase 2, thỏa yêu cầu có ít nhất một của Phase 1. Kiểm thử: T02A
BR10	Ngân sách còn lại không âm	CHECK remaining_budget>=0 và <=total_budget; total_budget>=0. Không suy diễn ngân sách từ bảng phân bổ không có chi phí. Kiểm thử: T07A

Mã	Quy tắc	Cơ chế và bằng chứng
BR11	Tài sản thuộc phòng được cấp cho đề tài	grant_laboratory và trigger allocation kiểm tra research_point→research_grant cùng lab tài sản. Không được xóa liên kết phòng hoặc chuyển hạng mục/di chuyển tài sản khi còn phân bổ. Kiểm thử: T10C,H08,H14
BR12	Completed/Suspended không nhận phân bổ mới	enforce_allocation khóa đề tài rồi yêu cầu Active; issue_chemical cũng yêu cầu đề tài Active. Phân bổ lịch sử được giữ khi đổi trạng thái. Kiểm thử: T10A,T10B
BR13	Không mượn tài sản bảo trì/ngừng dùng	Trigger khóa equipment và chỉ chấp nhận Available; unique partial index chặn hai lượt mượn chưa trả cho cùng thiết bị. Kiểm thử: T08A,T08B
BR14	Tối đa ba thiết bị chưa trả	Trigger ghi khóa người trước khi đếm actual_return_date IS NULL, nên Overdue chưa trả vẫn được tính. No-op UPDATE tạo xung đột ghi để mức cô lập mạnh hơn có thể hủy giao dịch xung đột. Kiểm thử: T08C; T14 cần PostgreSQL nhiều phiên
BR15	Trả trễ tự chuyển Overdue	enforce_borrow gán trạng thái từ ngày trả thực tế hoặc ngày hiện tại; v_open_loans thể hiện đúng trạng thái khi thời gian trôi. Kiểm thử: T09
BR16	Mỗi thiết bị chỉ thuộc một phòng	equipment.lab_id NOT NULL là một FK đơn tới laboratory. Kiểm thử: DDL,seed
BR17	Tồn hóa chất không âm	CHECK số tồn và atomic UPDATE có điều kiện quantity_in_stock>=quantity trong issue_chemical. Kiểm thử: T07B,H05
BR18	Không xuất hóa chất hết hạn	issue_chemical kiểm tra expiry_date<CURRENT_DATE sau khi khóa hóa chất. Kiểm thử: H04
BR19	Mọi thay đổi số tồn có kiểm soát	Tồn ban đầu bằng 0; Receipt/Issue cùng giao dịch với thay đổi số dư và ledger append-only. App, manager không được UPDATE số tồn hoặc DML ledger. Kiểm thử: H06,H07,H10,S01–S07
BR20	Extreme chỉ ở Hazardous	Trigger kiểm tra cả khi lưu hóa chất và khi hạ mức an toàn phòng; khóa hàng laboratory để chống thay đổi đồng thời. Kiểm thử: T06A,T06B
BR21	Ghi đủ thông tin truy cập và giao dịch	FK NOT NULL, entry_time và purpose không rỗng; exit_time cho phép NULL khi còn trong phòng. Danh tính lượt truy cập bất biến, đóng lượt qua close_access. Kiểm thử: Seed,DDL,T03B
BR22	Giờ ra sau giờ vào	CHECK exit_time IS NULL OR exit_time>entry_time. Kiểm thử: T12A

Mã	Quy tắc	Cơ chế và bằng chứng
BR23	Người hợp lệ mới tạo lượt vào khu nguy hiểm	Trigger yêu cầu person Active, chứng nhận hợp lệ và phê duyệt ngoài giờ; app không được INSERT access_log trực tiếp mà gọi record_access. Auth và xác nhận approval thuộc backend. Kiểm thử: T04,T05C
BR24	PK không NULL và duy nhất	PRIMARY KEY trên mọi bảng, PK ghép ở các quan hệ M:N; email, student_code, employee_code, serial_number có UNIQUE bổ sung. Kiểm thử: T11A,T11B
BR25	FK hợp lệ và bảo toàn lịch sử	Tất cả FK dùng ON DELETE/UPDATE RESTRICT. Ledger/lượt truy cập/mượn/bảo trì không được DELETE qua role nghiệp vụ. Kiểm thử: T02A,T02B,T13A,T13B
BR26	Ngày tháng hợp lệ	DATE/TIMESTAMP, NOT NULL; CHECK thứ tự các cặp ngày đề tài/hạng mục/chứng nhận/mượn/bảo trì. Kiểm thử: T05A,T12A,T12B

5.1.2 Thao tác kiểm thử và kết quả thực tế

expect_reject chỉ ghi đạt khi câu lệnh bị chặn và thông báo khớp điều kiện dự kiến; expect_true chỉ ghi đạt khi mệnh đề kiểm tra trả TRUE. Các ca dương tính và âm tính đều chạy trong một giao dịch được rollback ở cuối.

Mã	Câu lệnh kiểm tra	Kết quả thực tế
H01	SELECT issue_chemical(1,4,1,1,'Sai loại chứng nhận')	Đạt: BR06: all matching valid certificates required
H02	SELECT issue_chemical(2,5,1,0.1,'Thiếu chứng nhận axit')	Đạt: BR06: all matching valid certificates required
H03	SELECT issue_chemical(1,12,1,1,'Chứng nhận dung môi hết hạn')	Đạt: BR06: all matching valid certificates required
H04	SELECT issue_chemical(5,5,1,0.1,'Hóa chất hết hạn')	Đạt: BR18: expired chemical cannot be issued
H05	SELECT issue_chemical(1,5,1,999,'Vượt tồn kho')	Đạt: BR17: insufficient chemical stock
H06	SELECT issue_chemical(1,5,1,0,'Số lượng bằng không')	Đạt: BR19: positive quantity with at most 3 decimals required
H07	UPDATE chemical_movement SET quantity=quantity+1 WHERE movement_id=1	Đạt: BR19/21: history is append-only; use controlled workflow
H08	DELETE FROM grant_laboratory WHERE grant_id=1 AND lab_id=3	Đạt: BR11: remove allocations before grant lab membership

Mã	Câu lệnh kiểm tra	Kết quả thực tế
H09	SELECT quantity_in_stock=4.4 FROM chemical WHERE chemical_id=2	Đạt: Assertion true
H10	SELECT bool_and(reconciled) FROM v_chemical_stock_audit	Đạt: Assertion true
H11	SELECT issue_chemical(3,8,2,1,'Người ngừng hoạt động')	Đạt: BR06: active person required
H12	SELECT start_maintenance(4,12,'Sai vai trò',1)	Đạt: BR03: active technician required

Mã	Câu lệnh kiểm tra	Kết quả thực tế
H14	UPDATE equipment SET lab_id=3 WHERE equipment_id=1	Đạt: BR11: remove equipment allocations before lab relocation
T01A	INSERT INTO person(person_id,full_name,email) VALUES(900,'Không lớp con','no-subtype@test.edu')	Đạt: BR01: person 900 must have exactly one subtype
T01B	INSERT INTO faculty VALUES(4,'CNTT','Thạc sĩ',NULL)	Đạt: BR01: person 4 must have exactly one subtype
T01C	SELECT count(*)=1 FROM student WHERE student_id=901	Đạt: Assertion true
T01D	UPDATE faculty SET faculty_id=4 WHERE faculty_id=1	Đạt: BR01: person and subtype identifiers are immutable
T02A	INSERT INTO research_grant(grant_title,principal_investigator_id,start_date,end_date,total_budget,remaining_budget) VALUES('PI không hợp lệ',4,CURRENT_DATE,CURRENT_DATE+1,1,1)	Đạt: insert or update on table "research_grant" violates foreign key constraint "research_grant_principal_investigator_id_fkey"
T02B	INSERT INTO maintenance_log(technician_id,equipment_id,start_date,end_date,description,cost) VALUES(4,12,CURRENT_DATE,CURRENT_DATE,'Sai vai trò',1)	Đạt: insert or update on table "maintenance_log" violates foreign key constraint "maintenance_log_technician_id_fkey"
T03A	INSERT INTO access_log(person_id,lab_id,entry_time,purpose) VALUES(4,2,CURRENT_DATE+time '18:00','Thử truy cập')	Đạt: BR04: student out-of-hours approval required
T03B	SELECT approved_by=2 FROM access_log WHERE log_id=900	Đạt: Assertion true

Mã	Câu lệnh kiểm tra	Kết quả thực tế
T03C	INSERT INTO access_log(person_id,lab_id,entry_time,purpose,approved_by) VALUES(4,2,CURRENT_DATE+time '18:00','Phê duyệt sai',5)	Đạt: BR04: approval must be from another active faculty or technician
T03D	INSERT INTO access_log(person_id,lab_id,entry_time,purpose) VALUES(4,2,CURRENT_DATE+time '17:00','Mốc kết thúc giờ hành chính')	Đạt: BR04: student out-of-hours approval required
T04	INSERT INTO access_log(person_id,lab_id,entry_time,purpose) VALUES(7,3,CURRENT_DATE+time '09:00','Thiếu chứng nhận')	Đạt: BR05: valid safety certification required

Mã	Câu lệnh kiểm tra	Kết quả thực tế
T05A	UPDATE has_certification SET expiry_date=issue_date WHERE person_id=6 AND cert_id=1	Đạt: new row for relation "has_certification" violates check constraint "has_certification_check"
T05B	SELECT status='Expired' FROM has_certification WHERE person_id=6 AND cert_id=1	Đạt: Assertion true
T05C	INSERT INTO access_log(person_id,lab_id,entry_time,purpose) VALUES(6,2,CURRENT_DATE+time '09:00','Chứng nhận hết hạn')	Đạt: BR05: valid safety certification required
T06A	UPDATE chemical SET lab_id=1 WHERE chemical_id=2	Đạt: BR20: Extreme requires Hazardous lab
T06B	UPDATE laboratory SET safety_level='Normal' WHERE lab_id=3	Đạt: BR20: lab contains Extreme chemical
T07A	UPDATE research_grant SET remaining_budget=-1 WHERE grant_id=1	Đạt: new row for relation "research_grant" violates check constraint "research_grant_check"
T07B	UPDATE chemical SET quantity_in_stock=-1 WHERE chemical_id=3	Đạt: new row for relation "chemical" violates check constraint "chemical_quantity_in_stock_check"
T08A	SELECT checkout_equipment(4,6,CURRENT_DATE+3)	Đạt: BR13: equipment unavailable: Under Maintenance
T08B	SELECT checkout_equipment(4,9,CURRENT_DATE+3)	Đạt: BR13: equipment unavailable: Retired

Mã	Câu lệnh kiểm tra	Kết quả thực tế
T08C	SELECT checkout_equipment(4,11,CURRENT_DATE+3)	Đạt: BR14: maximum three open loans
T09	SELECT status='Overdue' AND actual_return_date=CURRENT_DATE FROM borrow_transaction WHERE borrow_id=1	Đạt: Assertion true
T10A	INSERT INTO allocated_equip VALUES(5,12)	Đạt: BR12: allocation requires Active grant

Mã	Câu lệnh kiểm tra	Kết quả thực tế
T10B	INSERT INTO allocated_chem VALUES(6,1)	Đạt: BR12: allocation requires Active grant
T10C	INSERT INTO allocated_equip VALUES(1,12)	Đạt: BR11: asset lab is not assigned to grant
T11A	INSERT INTO has_certification VALUES(4,1,CURRENT_DATE-1,CURRENT_DATE+300,'Valid')	Đạt: duplicate key value violates unique constraint "has_certification_pkey"
T11B	INSERT INTO allocated_equip VALUES(3,1)	Đạt: duplicate key value violates unique constraint "allocated_equip_pkey"
T12A	INSERT INTO access_log(person_id,lab_id,entry_time,exit_time,purpose) VALUES(4,1,CURRENT_DATE+time '09:00',CURRENT_DATE+time '08:00','Sai giờ')	Đạt: new row for relation "access_log" violates check constraint "access_log_check"
T12B	UPDATE research_grant SET end_date=start_date WHERE grant_id=1	Đạt: new row for relation "research_grant" violates check constraint "research_grant_check1"
T13A	DO $body$ BEGIN DELETE FROM has_certification WHERE person_id=4; DELETE FROM student WHERE student_id=4; DELETE FROM person WHERE person_id=4; END $body$	Đạt: update or delete on table "person" violates RESTRICT setting of foreign key constraint "access_log_person_id_fkey" on table "access_log"
T13B	DO $body$ BEGIN DELETE FROM allocated_equip WHERE equipment_id=5; DELETE FROM equipment WHERE equipment_id=5; END $body$	Đạt: update or delete on table "equipment" violates RESTRICT setting of foreign key constraint "borrow_transaction_equipment_id_fkey" on table "borrow_transaction"

5.1.3 Tập lệnh kiểm thử đầy đủ

```
SET search_path = lab_tracker, pg_catalog;

BEGIN;

CREATE TEMP TABLE phase3_test_result (
    test_id text PRIMARY KEY,
    passed boolean,
    actual text
);

CREATE FUNCTION pg_temp.expect_reject(p_id text, p_sql text, p_pattern text)
RETURNS void
LANGUAGE plpgsql
AS $$
DECLARE
    v_message text;
    v_rejected boolean := false;
BEGIN
    BEGIN
        EXECUTE p_sql;
        SET CONSTRAINTS ALL IMMEDIATE;
    EXCEPTION
        WHEN OTHERS THEN
            GET STACKED DIAGNOSTICS v_message = MESSAGE_TEXT;
            v_rejected := true;
    END;
    IF NOT v_rejected THEN
        RAISE EXCEPTION 'TEST % FAILED: statement was accepted', p_id;
    END IF;
    IF position(lower(p_pattern) IN lower(v_message)) = 0 THEN
        RAISE EXCEPTION 'TEST % FAILED: unexpected rejection %', p_id, v_message;
    END IF;
    INSERT INTO phase3_test_result VALUES (p_id, true, v_message);
END
$$;

CREATE FUNCTION pg_temp.expect_true(p_id text, p_sql text)
RETURNS void
LANGUAGE plpgsql
AS $$
DECLARE
    v_ok boolean;
BEGIN
    EXECUTE p_sql INTO v_ok;
    IF v_ok IS DISTINCT FROM true THEN
        RAISE EXCEPTION 'TEST % FAILED', p_id;
    END IF;
    INSERT INTO phase3_test_result VALUES (p_id, true, 'Assertion true');
END
$$;

SELECT pg_temp.expect_reject('T01A',
    $sql$INSERT INTO person(person_id,full_name,email) VALUES(900,'Không lớp con','no-
subtype@test.edu')$sql$,
     'BR01');
```

```
SELECT pg_temp.expect_reject('T01B',
    $sql$INSERT INTO faculty VALUES(4,'CNTT','Thạc sĩ',NULL)$sql$, 'BR01');

SELECT pg_temp.expect_reject('T01D',
    $sql$UPDATE faculty SET faculty_id=4 WHERE faculty_id=1$sql$, 'BR01');

INSERT INTO person(person_id, full_name, email)
VALUES(901,'Đúng một lớp con','one-subtype@test.edu');

INSERT INTO student
VALUES(901,'TEST901','CNTT',1);

SET CONSTRAINTS ALL IMMEDIATE;

SELECT pg_temp.expect_true('T01C',
    $sql$SELECT count(*)=1 FROM student WHERE student_id=901$sql$);

SET CONSTRAINTS ALL DEFERRED;

SELECT pg_temp.expect_reject('T02A',
    $sql$INSERT INTO
research_grant(grant_title,principal_investigator_id,start_date,end_date,total_budget,r
emaining_budget) VALUES('PI không hợp lệ',4,CURRENT_DATE,CURRENT_DATE+1,1,1)$sql$,
     'foreign key');

SELECT pg_temp.expect_reject('T02B',
    $sql$INSERT INTO
maintenance_log(technician_id,equipment_id,start_date,end_date,description,cost)
VALUES(4,12,CURRENT_DATE,CURRENT_DATE,'Sai vai trò',1)$sql$,
     'foreign key');

SELECT pg_temp.expect_reject('T03A',
    $sql$INSERT INTO access_log(person_id,lab_id,entry_time,purpose)
VALUES(4,2,CURRENT_DATE+time '18:00','Thử truy cập')$sql$,
     'BR04');

INSERT INTO access_log(log_id,
                person_id, lab_id, entry_time, purpose, approved_by)
VALUES(900,4,2,CURRENT_DATE + time '18:00','Thử truy cập',2);

SELECT pg_temp.expect_true('T03B',
    $sql$SELECT approved_by=2 FROM access_log WHERE log_id=900$sql$);

SELECT pg_temp.expect_reject('T03C',
    $sql$INSERT INTO access_log(person_id,lab_id,entry_time,purpose,approved_by)
VALUES(4,2,CURRENT_DATE+time '18:00','Phê duyệt sai',5)$sql$,
     'BR04');

SELECT pg_temp.expect_reject('T03D',
    $sql$INSERT INTO access_log(person_id,lab_id,entry_time,purpose)
VALUES(4,2,CURRENT_DATE+time '17:00','Mốc kết thúc giờ hành chính')$sql$,
     'BR04');

SELECT pg_temp.expect_reject('T04',
    $sql$INSERT INTO access_log(person_id,lab_id,entry_time,purpose)
VALUES(7,3,CURRENT_DATE+time '09:00','Thiếu chứng nhận')$sql$,
     'BR05');
```

```
SELECT pg_temp.expect_reject('T05A',
    $sql$UPDATE has_certification SET expiry_date=issue_date WHERE person_id=6 AND
cert_id=1$sql$,
     'check constraint');

UPDATE has_certification
SET status = 'Valid'
WHERE person_id = 6
    AND cert_id = 1;

SELECT pg_temp.expect_true('T05B',
    $sql$SELECT status='Expired' FROM has_certification WHERE person_id=6 AND
cert_id=1$sql$);

SELECT pg_temp.expect_reject('T05C',
    $sql$INSERT INTO access_log(person_id,lab_id,entry_time,purpose)
VALUES(6,2,CURRENT_DATE+time '09:00','Chứng nhận hết hạn')$sql$,
     'BR05');

SELECT pg_temp.expect_reject('T06A',
    $sql$UPDATE chemical SET lab_id=1 WHERE chemical_id=2$sql$, 'BR20');

SELECT pg_temp.expect_reject('T06B',
    $sql$UPDATE laboratory SET safety_level='Normal' WHERE lab_id=3$sql$, 'BR20');

SELECT pg_temp.expect_reject('T07A',
    $sql$UPDATE research_grant SET remaining_budget=-1 WHERE grant_id=1$sql$,
    'check constraint');

SELECT pg_temp.expect_reject('T07B',
    $sql$UPDATE chemical SET quantity_in_stock=-1 WHERE chemical_id=3$sql$,
    'check constraint');

SELECT pg_temp.expect_reject('T08A',
    $sql$SELECT checkout_equipment(4,6,CURRENT_DATE+3)$sql$, 'BR13');

SELECT pg_temp.expect_reject('T08B',
    $sql$SELECT checkout_equipment(4,9,CURRENT_DATE+3)$sql$, 'BR13');

SELECT checkout_equipment(4, 3, CURRENT_DATE + 3);

SELECT pg_temp.expect_reject('T08C',
    $sql$SELECT checkout_equipment(4,11,CURRENT_DATE+3)$sql$, 'BR14');

UPDATE borrow_transaction
SET actual_return_date = CURRENT_DATE
WHERE borrow_id = 1;

SELECT pg_temp.expect_true('T09',
    $sql$SELECT status='Overdue' AND actual_return_date=CURRENT_DATE FROM
borrow_transaction WHERE borrow_id=1$sql$);

SELECT pg_temp.expect_reject('T10A',
    $sql$INSERT INTO allocated_equip VALUES(5,12)$sql$, 'BR12');

SELECT pg_temp.expect_reject('T10B',
    $sql$INSERT INTO allocated_chem VALUES(6,1)$sql$, 'BR12');
```

```
SELECT pg_temp.expect_reject('T10C',
    $sql$INSERT INTO allocated_equip VALUES(1,12)$sql$, 'BR11');

SELECT pg_temp.expect_reject('T11A',
    $sql$INSERT INTO has_certification VALUES(4,1,CURRENT_DATE-
1,CURRENT_DATE+300,'Valid')$sql$,
     'duplicate key');

SELECT pg_temp.expect_reject('T11B',
    $sql$INSERT INTO allocated_equip VALUES(3,1)$sql$, 'duplicate key');

SELECT pg_temp.expect_reject('T12A',
    $sql$INSERT INTO access_log(person_id,lab_id,entry_time,exit_time,purpose)
VALUES(4,1,CURRENT_DATE+time '09:00',CURRENT_DATE+time '08:00','Sai giờ')$sql$,
     'check constraint');

SELECT pg_temp.expect_reject('T12B',
    $sql$UPDATE research_grant SET end_date=start_date WHERE grant_id=1$sql$,
    'check constraint');

SELECT pg_temp.expect_reject('T13A',
    $sql$DO $body$ BEGIN DELETE FROM has_certification WHERE person_id=4; DELETE FROM
student WHERE student_id=4; DELETE FROM person WHERE person_id=4; END $body$$sql$,
     'foreign key');

SELECT pg_temp.expect_reject('T13B',
    $sql$DO $body$ BEGIN DELETE FROM allocated_equip WHERE equipment_id=5; DELETE FROM
equipment WHERE equipment_id=5; END $body$$sql$,
     'foreign key');

SELECT pg_temp.expect_reject('H01',
    $sql$SELECT issue_chemical(1,4,1,1,'Sai loại chứng nhận')$sql$, 'BR06');

SELECT pg_temp.expect_reject('H02',
    $sql$SELECT issue_chemical(2,5,1,0.1,'Thiếu chứng nhận axit')$sql$, 'BR06');

SELECT pg_temp.expect_reject('H03',
    $sql$SELECT issue_chemical(1,12,1,1,'Chứng nhận dung môi hết hạn')$sql$,
    'BR06');

SELECT pg_temp.expect_reject('H04',
    $sql$SELECT issue_chemical(5,5,1,0.1,'Hóa chất hết hạn')$sql$, 'BR18');

SELECT pg_temp.expect_reject('H05',
    $sql$SELECT issue_chemical(1,5,1,999,'Vượt tồn kho')$sql$, 'BR17');

SELECT pg_temp.expect_reject('H06',
    $sql$SELECT issue_chemical(1,5,1,0,'Số lượng bằng không')$sql$, 'BR19');

SELECT pg_temp.expect_reject('H07',
    $sql$UPDATE chemical_movement SET quantity=quantity+1 WHERE movement_id=1$sql$,
    'append-only');

SELECT pg_temp.expect_reject('H08',
    $sql$DELETE FROM grant_laboratory WHERE grant_id=1 AND lab_id=3$sql$, 'BR11');

SELECT issue_chemical(2,
           10, 1, 0.1, 'Kiểm thử cấp Extreme hợp lệ');
```

```
SELECT pg_temp.expect_true('H09',
    $sql$SELECT quantity_in_stock=4.4 FROM chemical WHERE chemical_id=2$sql$);

SELECT pg_temp.expect_true('H10',
    $sql$SELECT bool_and(reconciled) FROM v_chemical_stock_audit$sql$);

SELECT pg_temp.expect_reject('H11',
    $sql$SELECT issue_chemical(3,8,2,1,'Người ngừng hoạt động')$sql$, 'BR06');

SELECT pg_temp.expect_reject('H12',
    $sql$SELECT start_maintenance(4,12,'Sai vai trò',1)$sql$, 'BR03');

SELECT pg_temp.expect_reject('H14',
    $sql$UPDATE equipment SET lab_id=3 WHERE equipment_id=1$sql$, 'BR11');

SELECT test_id,passed,actual
FROM phase3_test_result
ORDER BY test_id;

ROLLBACK;
```

5.2 Phân quyền truy cập theo vai trò

Các role nghiệp vụ đều NOLOGIN; tài khoản đăng nhập của backend nhận đúng role cần thiết khi triển khai. lab_reader đọc danh mục và view quản trị. lab_app được gọi các hàm truy cập, mượn trả và xuất hóa chất. lab_technician kế thừa lab_app, được nhập hóa chất và thao tác bảo trì. lab_manager quản lý danh mục, chứng nhận, đề tài và phân bổ nhưng không được sửa số tồn hoặc chèn ledger.

REVOKE mặc định quyền PUBLIC trên schema, bảng, sequence và hàm. Hàm SECURITY DEFINER có search_path cố định lab_tracker,pg_catalog,pg_temp, nên quan hệ tạm do người gọi tạo không thể che bảng nghiệp vụ. Các role chỉ có USAGE, không có CREATE trên schema. Quyền execute được cấp theo chữ ký hàm cụ thể.

p_actor và p_approver là các tham số được backend tin cậy cung cấp. SQL xác minh trạng thái, lớp con và điều kiện nghiệp vụ; SQL không xác thực mật khẩu, phiên đăng nhập hoặc bằng chứng người duyệt đã thật sự chấp thuận. Phase 4 phải lấy actor từ phiên xác thực và approver từ bản ghi yêu cầu đã duyệt, không nhận các giá trị này trực tiếp từ nội dung yêu cầu do trình duyệt gửi. Không công khai tài khoản database cho người dùng cuối.

Chủ sở hữu cơ sở dữ liệu và superuser chỉ phục vụ cài đặt, di trú và bảo trì được kiểm soát. Kiểm thử SET ROLE chứng minh tài khoản nghiệp vụ và manager không thể tự sửa quantity_in_stock hoặc giả mạo chemical_movement.

5.2.1 Câu lệnh GRANT và REVOKE

```
CREATE ROLE lab_reader NOLOGIN;

CREATE ROLE lab_app NOLOGIN;

CREATE ROLE lab_technician NOLOGIN;

CREATE ROLE lab_manager NOLOGIN;

GRANT lab_app TO lab_technician;

GRANT lab_reader TO lab_manager;

GRANT USAGE ON SCHEMA lab_tracker TO lab_reader, lab_app, lab_technician,
    lab_manager;

GRANT SELECT ON lab_tracker.v_open_loans, lab_tracker.v_grant_asset_summary,
    lab_tracker.v_chemical_stock_audit, lab_tracker.v_certification_current TO
    lab_reader, lab_app;

GRANT SELECT ON lab_tracker.laboratory, lab_tracker.equipment, lab_tracker.chemical,
    lab_tracker.certification, lab_tracker.research_grant,
    lab_tracker.research_point TO lab_reader, lab_app;

GRANT EXECUTE ON FUNCTION lab_tracker.issue_chemical(bigint, bigint, bigint,
    numeric, text), lab_tracker.checkout_equipment(bigint, bigint, date),
    lab_tracker.return_equipment(bigint, bigint), lab_tracker.record_access(bigint,
    bigint, text, bigint), lab_tracker.close_access(bigint, bigint) TO lab_app;

GRANT EXECUTE ON FUNCTION lab_tracker.receive_chemical(bigint, bigint, numeric,
    text), lab_tracker.start_maintenance(bigint, bigint, text, numeric),
    lab_tracker.finish_maintenance(bigint, bigint, text, numeric) TO lab_technician;

GRANT SELECT ON ALL TABLES IN SCHEMA lab_tracker TO lab_manager;

GRANT INSERT, UPDATE, DELETE ON lab_tracker.person, lab_tracker.faculty,
    lab_tracker.student, lab_tracker.technician, lab_tracker.laboratory,
    lab_tracker.research_grant, lab_tracker.research_point,
    lab_tracker.certification, lab_tracker.has_certification,
    lab_tracker.allocated_equip, lab_tracker.allocated_chem,
    lab_tracker.grant_laboratory, lab_tracker.chemical_certification TO lab_manager;

GRANT INSERT ON lab_tracker.equipment, lab_tracker.chemical TO lab_manager;

GRANT UPDATE(lab_id, equipment_name, serial_number, purchase_date, purchase_cost) ON
    lab_tracker.equipment TO lab_manager;

GRANT UPDATE(lab_id, chemical_name, cas_number, unit, expiry_date, hazard_level,
    storage_condition) ON lab_tracker.chemical TO lab_manager;

GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA lab_tracker TO lab_manager;

GRANT EXECUTE ON FUNCTION lab_tracker.refresh_temporal_status() TO lab_manager;

REVOKE INSERT, UPDATE, DELETE ON lab_tracker.chemical_movement FROM lab_reader,
    lab_app, lab_technician, lab_manager;
```

```
REVOKE UPDATE(quantity_in_stock) ON lab_tracker.chemical FROM lab_reader, lab_app,
    lab_technician, lab_manager;

REVOKE INSERT, UPDATE, DELETE ON lab_tracker.access_log,
    lab_tracker.borrow_transaction, lab_tracker.maintenance_log FROM lab_reader,
    lab_app, lab_technician, lab_manager;

ALTER ROLE lab_reader
SET search_path = lab_tracker,pg_catalog;

ALTER ROLE lab_app
SET search_path = lab_tracker,pg_catalog;

ALTER ROLE lab_technician
SET search_path = lab_tracker,pg_catalog;

ALTER ROLE lab_manager
SET search_path = lab_tracker,pg_catalog;

ALTER ROLE lab_reader
SET timezone = 'Asia/Ho_Chi_Minh';

ALTER ROLE lab_app
SET timezone = 'Asia/Ho_Chi_Minh';

ALTER ROLE lab_technician
SET timezone = 'Asia/Ho_Chi_Minh';

ALTER ROLE lab_manager
SET timezone = 'Asia/Ho_Chi_Minh';
```

5.2.2 Kết quả kiểm thử quyền

Mã	Vai trò	Kỳ vọng	Kết quả thực tế
S01	lab_app	Từ chối	Đạt: permission denied for table chemical (42501)
S02	lab_manager	Từ chối	Đạt: permission denied for table chemical (42501)
S03	lab_app	Từ chối	Đạt: permission denied for table chemical_movement (42501)
S04	lab_app	Cho phép	Đạt: Accepted
S05	lab_app	Từ chối	Đạt: permission denied for function receive_chemical (42501)
S06	lab_technician	Cho phép	Đạt: Accepted
S07	lab_reader	Từ chối	Đạt: permission denied for function issue_chemical (42501)
S08	lab_app	Từ chối	Đạt: permission denied for table maintenance_log (42501)
S09	lab_technician	Từ chối	Đạt: BR03: active technician required (P0001)

Mã	Vai trò	Kỳ vọng	Kết quả thực tế
S10	lab_app	Cho phép	Đạt: Accepted

Phụ lục A Phần bổ sung của lược đồ vật lý

Phase 2, mục 3.1–3.16 đã mô tả từng thuộc tính, kiểu dữ liệu, khóa, khả năng NULL, mặc định và ý nghĩa của 16 bảng nền. Phụ lục này tham chiếu các bảng đó và chỉ ghi quyết định thực thi bổ sung, cùng từ điển đầy đủ cho ba bảng mới. NUMERIC và DECIMAL là hai tên tương đương trong PostgreSQL; các độ chính xác được giữ nguyên. Cột cho phép NULL mà không có DEFAULT vẫn nhận NULL khi bị bỏ qua trong INSERT. Toàn bộ PK, FK, CHECK, chỉ mục, view, trigger và hàm vẫn có mã đầy đủ tại mục 4.1.5; truy vết BR01–BR26 và bằng chứng kiểm thử ở mục 5.1.1.

A.1 Các quyết định vật lý bổ sung cho 16 bảng nền

Phạm vi	Bổ sung trong triển khai PostgreSQL
Toàn bộ bảng nền	Giữ tên cột, PK/FK và khả năng NULL theo Phase 2; mọi FK dùng ON UPDATE RESTRICT và ON DELETE RESTRICT. Mười khóa chính sự kiện/thực thể dùng GENERATED BY DEFAULT AS IDENTITY: person_id, lab_id, grant_id, rearch_point_id, equipment_id, chemical_id, cert_id, log_id, borrow_id và maintenance_id. Khóa lớp con kế thừa person_id; các liên kết M:N giữ PK ghép.
person; faculty; student; technician	full_name không được chỉ có khoảng trắng; email có CHECK định dạng và unique index trên lower(email). student_code và employee_code có UNIQUE. Constraint trigger trì hoãn yêu cầu đúng một lớp con lúc commit; mã người/lớp con bất biến.
laboratory	UNIQUE(building,room_number); trigger từ chối hạ mức an toàn của phòng còn chứa hóa chất Extreme.
research_grant	CHECK bổ sung remaining_budget <= total_budget; cấp phòng cho đề tài qua grant_laboratory. Trạng thái đề tài được khóa/kiểm tra khi phân bổ hoặc xuất hóa chất.
research_point	Miền status chốt thành Planned, Active, Completed hoặc Suspended. Trigger không cho đổi grant_id khi hạng mục còn tài sản phân bổ.
equipment	serial_number có UNIQUE. Trigger bảo vệ chuyển phòng khi còn phân bổ và phối hợp trạng thái với các hàm mượn/trả/bảo trì.
chemical	quantity_in_stock có DEFAULT 0; unit giới hạn g, kg, mL, L, mol. Chỉ receive_chemical và issue_chemical thay đổi tồn qua giao dịch có ledger; hóa chất Extreme phải thuộc phòng Hazardous.

Phạm vi	Bổ sung trong triển khai PostgreSQL
certification; has_certification	cert_name có UNIQUE. sync_cert_status chuẩn hóa trạng thái khi ghi; refresh_temporal_status và view/hàm nghiệp vụ kiểm tra ngày hiệu lực. Index(person_id,expiry_date) hỗ trợ tra chứng nhận.
allocated_equip; allocated_chem	Giữ nguyên hai PK ghép và không thêm số lượng/chi phí. Trigger kiểm tra đề tài Active và phòng tài sản thuộc grant_laboratory; bảo vệ liên kết khi di chuyển tài sản hoặc đổi phòng đề tài.
access_log	purpose phải có nội dung; danh tính lượt truy cập bất biến. record_access/close_access kiểm soát mở/đóng lượt, trạng thái người, chứng nhận và phê duyệt ngoài giờ.
borrow_transaction	Partial unique index(equipment_id) WHERE actual_return_date IS NULL chặn hai lượt giữ cùng thiết bị. Trigger khóa người/thiết bị, giới hạn ba lượt chưa trả, kiểm tra trạng thái và chuẩn hóa Returned/Overdue.
maintenance_log	Partial unique index(equipment_id) WHERE end_date IS NULL chặn hai bảo trì mở. start_maintenance/finish_maintenance kiểm tra Technician Active và phối hợp trạng thái thiết bị; role nghiệp vụ không DML trực tiếp bảng.

Các bảng A.2–A.4 sau đây lấy metadata từ schema thực thi. PK là khóa chính, FK là khóa ngoại; dấu gạch ngang biểu thị cột không thuộc khóa. DEFAULT Không nghĩa là DDL không khai báo mặc định; khóa tự sinh dùng IDENTITY. Mỗi FK sử dụng ON UPDATE RESTRICT và ON DELETE RESTRICT.

A.2 Bảng grant_laboratory

Thuộc tính	Kiểu dữ liệu	Khóa	NULL	Mặc định	Ràng buộc
grant_id	BIGINT	FK, PK	Không	Không	FOREIGN KEY (grant_id) REFERENCES research_grant(grant_id) ON UPDATE RESTRICT ON DELETE RESTRICT
lab_id	BIGINT	FK, PK	Không	Không	FOREIGN KEY (lab_id) REFERENCES laboratory(lab_id) ON UPDATE RESTRICT ON DELETE RESTRICT

A.3 Bảng chemical_certification

Thuộc tính	Kiểu dữ liệu	Khóa	NULL	Mặc định	Ràng buộc
chemical_id	BIGINT	FK, PK	Không	Không	FOREIGN KEY (chemical_id) REFERENCES chemical(chemical_id) ON UPDATE RESTRICT ON DELETE RESTRICT
cert_id	BIGINT	FK, PK	Không	Không	FOREIGN KEY (cert_id) REFERENCES certification(cert_id) ON UPDATE RESTRICT ON DELETE RESTRICT

A.4 Bảng chemical_movement

Thuộc tính	Kiểu dữ liệu	Khóa	NULL	Mặc định	Ràng buộc
movement_id	BIGINT	PK	Không	Tự sinh IDENTITY	Theo miền dữ liệu và quy tắc nghiệp vụ tương ứng.
chemical_id	BIGINT	FK	Không	Không	FOREIGN KEY (chemical_id) REFERENCES chemical(chemical_id) ON UPDATE RESTRICT ON DELETE RESTRICT
person_id	BIGINT	FK	Không	Không	FOREIGN KEY (person_id) REFERENCES person(person_id) ON UPDATE RESTRICT ON DELETE RESTRICT
rearch_point_id	BIGINT	FK	Có	Không	CHECK (movement_type = 'Issue' AND rearch_point_id IS NOT NULL OR movement_type = 'Receipt' AND rearch_point_id IS NULL); FOREIGN KEY (rearch_point_id) REFERENCES research_point(rearch_point_id) ON UPDATE RESTRICT ON DELETE RESTRICT

Thuộc tính	Kiểu dữ liệu	Khóa	NULL	Mặc định	Ràng buộc
movement_type	VARCHAR (10)	-	Không	Không	CHECK (movement_type = ANY (ARRAY['Receipt', 'Issue'][])); CHECK (movement_type = 'Issue' AND rearch_point_id IS NOT NULL OR movement_type = 'Receipt' AND rearch_point_id IS NULL)
quantity	NUMERIC(14,3)	-	Không	Không	CHECK (quantity > 0::numeric)
occurred_at	TIMESTAMPTZ	-	Không	clock_timestamp()	Theo miền dữ liệu và quy tắc nghiệp vụ tương ứng.
reason	VARCHAR (500)	-	Không	Không	CHECK (btrim(reason) <> '')
balance_after	NUMERIC(14,3)	-	Không	Không	CHECK (balance_after >= 0::numeric)